

MSSD-SLAM: Multifeature Semantic RGB-D Inertial SLAM With Structural Regularity for Dynamic Environments

Yanan Wang^{1b}, Yaobin Tian^{1b}, Jiawei Chen^{1b}, *Member, IEEE*, Cheng Chen^{1b},
Kun Xu^{1b}, *Member, IEEE*, and Xilun Ding^{1b}

Abstract—Traditional visual simultaneous localization and mapping (SLAM) methods excel in static, texture-rich environments, but may struggle in dynamic, textureless environments. Recent advances in dynamic SLAM incorporate deep learning techniques to eliminate dynamic objects. However, these methods may suffer from unrecognizable dynamic objects, and removing dynamic feature points exacerbates feature scarcity. To address the limitations, we propose MSSD-SLAM, a multi-feature semantic RGB-D inertial SLAM system that incorporates point, line, and plane features to enhance robustness and enrich static map fidelity. By embedding structural constraints on 3-D spatial features constructed from multiframe observations, it guarantees geometric consistency and accurate camera pose estimation, supporting subsequent dynamic object detection. A novel Dynamic Filter has been developed that efficiently handles both semantically recognizable and unrecognizable objects by amalgamating semantic segmentation, structural constraints, and inertial measurement unit (IMU) measurements. Short-term dynamic objects are detected by the consistency of multisource information, while long-term dynamic objects, which may remain static temporarily, are identified through covisible projection among multiple frames. Validation on the TUM dataset and real-world scenarios demonstrates that the localization accuracy of MSSD-SLAM outperforms ORB-SLAM3 and Dynamic-VINS by 76% and 64%, respectively, indicating that our algorithm exhibits superior accuracy and robustness in dynamic indoor scenes compared with state-of-the-art algorithms.

Index Terms—Dynamic environment, line and plane features, RGB-D inertial simultaneous localization and mapping (SLAM), structural regularity.

NOMENCLATURE

Symbol	Description
$\mathbf{p}$	Two-dimensional point in image.
$\mathbf{P}$	Three-dimensional point in space.
$\mathbf{d}$	Direction of a 3-D line.

$\mathbf{n}$	Normal vector of a plane.
$\mathbf{L}$	Plücker coordinates of a 3-D line.
$\mathbf{L}^*$	Plücker matrix of a 3-D line.
$\mathbf{m}$	Unit normal vector of a plane.
π	Three-dimensional plane in space.
$\mathbf{T}$	Three-dimensional transformation matrix.
$\mathbf{l}$	Two-dimensional line in image.
d	Distance from a plane to the origin.
$\mathbf{H}$	Projection matrix.
$\mathbf{s}, \mathbf{e}$	Endpoints of 2-D line.
$\mathbf{S}, \mathbf{E}$	Endpoints of 3-D line.
$\mathbf{R}$	Three-dimensional rotation matrix.
$\mathbf{t}$	Three-dimensional translation vector.
$\mathbf{K}$	Camera intrinsic matrix.
$\mathbf{F}$	Fundamental matrix.

I. INTRODUCTION

VISUAL simultaneous localization and mapping (SLAM) has gained significant traction in various domains, including indoor service robots, autonomous driving, and virtual reality. While some visual SLAM algorithms, such as ORB-SLAM family [1], [2], [3] and VINS-Mono [4], exhibit outstanding performance in scenes with abundant textures, they face challenges in featureless environments. Moreover, the majority of visual SLAM algorithms assume a static scene, disregarding the existence of dynamic objects, such as pedestrians and vehicles in real-world scenarios.

The challenges of visual SLAM in low-texture scenes can be mitigated by combining point features with line features [5], [6], [7], [8], plane features [9], or a combination of both [10], [11], [12]. To address the visual SLAM problem in dynamic scenes, two main approaches are employed: geometry-based [13], [14], [15], [16], [17] and semantic-based [18], [19], [20], [21], [22], [23], [24], [25]. Geometry-based methods detect dynamic objects using constraints, such as epipolar geometry and multiview geometry, or by incorporating sensors, such as IMU, to ensure motion consistency. However, these geometry-based SLAM methods lack important semantic prior information, which limits their robustness in highly dynamic scenes or when dealing with temporarily stationary dynamic objects.

Received 18 March 2024; revised 8 July 2024; accepted 30 July 2024. Date of publication 2 December 2024; date of current version 17 December 2024. This work was supported by the National Natural Science Foundation of China under Grant 52375003, Grant U22B2080, and Grant T2121003. The Associate Editor coordinating the review process was Dr. Huan Liu. (*Corresponding author: Kun Xu.*)

The authors are with the Robotics Institute, School of Mechanical Engineering and Automation, Beihang University (BUAA), Beijing 100191, China (e-mail: wang_yanan@buaa.edu.cn; ybtian@buaa.edu.cn; chenjiawei@buaa.edu.cn; cheng_chen@buaa.edu.cn; xk007@buaa.edu.cn; xlding@buaa.edu.cn).

Data is available on-line at <https://www.youtube.com/watch?v=RU6tGEehM6Q>.

Digital Object Identifier 10.1109/TIM.2024.3509541

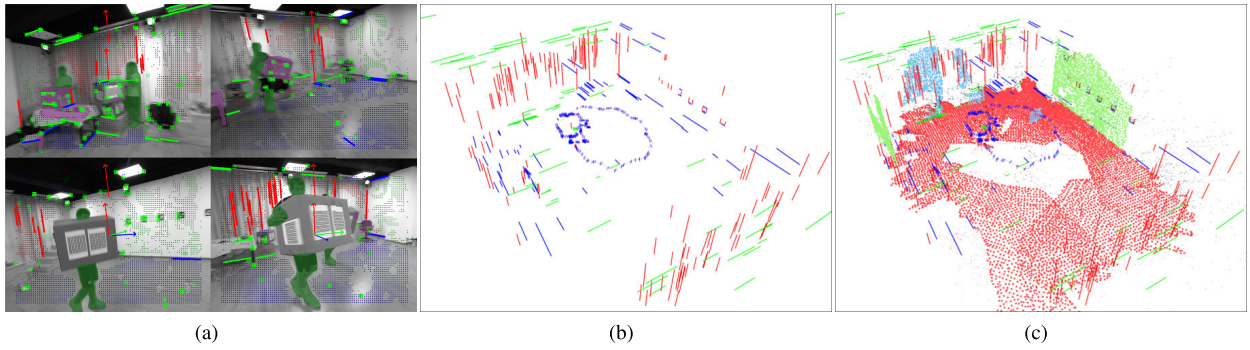


Fig. 1. Overview of MSSD-SLAM results. (a) Screenshots of the MSSD-SLAM run and dynamic features have been excluded by the Dynamic Filter. The coordinate system in the center of the figure displays the detected Manhattan world frame (MWF), while the red arrow above it presents the gravity direction derived from the IMU. (b) Static map with lines. (c) Static map with points, lines, and planes. Structured map lines are exhibited in red, green, and blue, respectively, aligned with the X -, Y -, and Z -axis directions, while red, green, and blue planes depict the environment planes where the normal vectors align with the X -, Y -, and Z -axis directions.

In contrast, semantic-based methods incorporate techniques, such as object detection and semantic segmentation, into SLAM, leveraging the object's semantic information as prior knowledge and combining it with geometric constraints to identify dynamic objects. Despite the extensive use of semantic-based methods in SLAM for dynamic scenes, these methods have limitations in detecting certain types of objects and integrating information from other sources, rendering them vulnerable to undetectable dynamic objects. Moreover, relying solely on feature points in SLAM methods may lead to failures in textureless dynamic indoor scenes, such as corridors. The removal of feature points associated with dynamic objects in the scene exacerbates the instability of SLAM tracking.

Our prior study, DRG-SLAM [26] presented a dynamic visual SLAM using RGB-D cameras. Nonetheless, purely visual approaches are vulnerable to interference from dynamic objects in the environment. Furthermore, the SegNet semantic segmentation method [27] used in DRG-SLAM has limitations in terms of accuracy and efficiency. This approach has considerable limitations in handling dynamic objects and is, thus, insufficient to handle moving objects undetectable by deep learning models, particularly in highly dynamic scenes.

Recognizing these challenges, we present MSSD-SLAM, a novel semantic visual-inertial SLAM method that integrates point, line, and plane features and combines inertial measurement unit (IMU) and visual data. Our approach uses multiple sources of information and incorporates them into the proposed multistage Dynamic Filter. Specifically, this filter recognizes both objects identified through semantic segmentation and those that cannot be recognized. In addition, our method can perform stable tracking in featureless scenes by filtering out dynamic features. Simultaneously, it constructs a static map comprising spatial points, lines, and planes (Fig. 1).

In summary, the main contributions of this article are articulated as follows.

- 1) *Robust RGB-D Inertial SLAM*: We propose a novel method that integrates semantic and geometric information with visual and IMU measurements. This approach enhances robustness in textureless environments and improves static map construction in indoor dynamic scenes.
- 2) *Structural Constraints on 3-D Features*: We introduce structural constraints for 3-D line and plane features based on the Manhattan world assumption. This improves geometric consistency and minimizes errors from dynamic object occlusion, aiding in accurate camera pose estimation and map fidelity.
- 3) *Innovative Dynamic Filter*: We develop a Dynamic Filter that fuses multisource information from semantic segmentation, structural constraints, and IMU observations. This filter accurately identifies both semantically identifiable and unidentifiable dynamic objects, improving robustness in dynamic scenes.
- 4) *Extensive Validation*: Our method has been extensively tested on the TUM dataset and real-world scenarios. The results show that MSSD-SLAM outperforms ORB-SLAM3 by 76% and Dynamic-VINS by 64% in localization accuracy, demonstrating enhanced performance in indoor dynamic scenes.

The remainder of this article is organized as follows. Section II presents a concise summary of the relevant studies. Section III provides an overview of the proposed method. Section IV explains the feature representation and structural regularity. Section V models the Dynamic Filter. Section VI validates the proposed approach through experiments and compares it with state-of-the-art methods. Finally, Section VII concludes this article and outlines future research directions.

II. RELATED WORKS

A. SLAM With Line or Plane Features

To enhance stability, many researchers have integrated line or plane features into the SLAM framework.

Zhang et al. [5] proposed a stereo visual SLAM method that utilizes just line features, representing 3-D lines using Plücker coordinates and building a 3-D line map. Conversely, Zuo et al. [28] use both point and line features in SLAM. In PL-VIO [6], line features are jointly optimized with point features in monocular visual-inertial odometry. PL-SLAM [7] integrates point and line features at multiple stages of SLAM, enhancing robustness in low-texture environments. Sun et al. [29] combine dynamic matching with median absolute deviation for matching line features and perform

pose estimation by jointly optimizing twin line and point reprojection errors.

Some researchers leverage scene structure information to simplify the representation of line features. StructVIO [8] simplifies the representation of structured lines by projecting them onto a line parameter space through a rotation matrix. Wei et al. [30] align the Manhattan world coordinate system with the global coordinate system and simplify the representation of lines and Jacobian calculation by representing these lines as 2-D points on the plane. In contrast, Xu et al. [31] consider both structured and nonstructured lines.

Planar features in the scene can offer distinct constraints. Zhang et al. [9] increase the number of plane observations and constraints in RGB-D SLAM by using the edges of planes for data association. Structure-SLAM [10] integrates point, line, and plane features and employs convolutional neural networks to predict surface normals for each input RGB image. By tracking the Manhattan world in the environment, PlanarSLAM [11] achieves drift-free rotation matrix estimation. Structure PLP-SLAM [12] utilizes point, line, and plane features. The authors initialize planes using a plane instance segmentation network and reconstruct planes by fitting 3-D points.

B. Visual SLAM in Dynamic Scenes

The aforementioned methods are built upon static scene assumptions, thereby restricting their applicability in real dynamic environments.

To counter the influence of dynamic objects, geometry-based approaches have been explored. Kim and Kim [13] leveraged multiview geometry for constructing a static background model, effectively diminishing the impact of dynamic entities. Li and Lee [14] utilized static weighting methods for mitigating dynamic object interference in pose estimation. Sun et al. [15] employed optical flow for identifying dynamic foreground elements. Du et al. [16] fused conditional random fields with multiframe long-term observations for dynamic object detection and tracking. In DynaVINS [17], a novel approach using bundle adjustment (BA) was introduced, excluding dynamic object features based on IMU preintegration-derived pose estimates.

While geometric methods are computationally efficient, they lack semantic priors, leading to challenges in detecting transient dynamic objects. This limitation has spurred the integration of semantic data into dynamic SLAM systems. DS-SLAM [18] augmented ORB-SLAM2 [2] with a semantic segmentation module to identify and eliminate dynamic points. DynaSLAM [19], merging instance segmentation with multiview geometry, facilitated dynamic object detection, a process further refined in DynaSLAM II [20] through dynamic object tracking. DRE-SLAM [21] utilized K -means clustering of depth images alongside multiview geometry constraints for dynamic object identification. Clustering-based approaches have been employed for dynamic object motion estimation. ClusterSLAM [32] and ClusterVO [33] focused on clustering dynamic landmarks and estimating their motion, distinguishing individual rigid bodies through their unique

motion patterns. SaD-SLAM [22] categorized feature points, processing each type distinctively within the SLAM algorithm. RDS-SLAM [23] adopted a strategy allowing the tracking thread to operate independently of the semantic segmentation thread. Dynamic-VINS [24] integrated IMU data for dynamic object detection, catering to platforms with constrained computational resources.

In our prior work, DRG-SLAM [26], we employed SegNet for semantic segmentation and combined epipolar geometry and multiview constraints for dynamic object determination, alongside point–line–plane features to bolster robustness. Pan et al. [34] developed a method using multiview geometry and Gaussian distribution for dynamic object identification and 3-D reconstruction. SG-SLAM [35] extended ORB-SLAM2 by integrating an object detection and semantic mapping module, with the tracking thread implementing a rapid dynamic feature removal algorithm that synergizes semantic and geometric data. DE-SLAM [36] merged dynamic object detection and tracking with loop closure detection, employing a lightweight object detection network and adaptive particle filtering. Chang et al. [37] enhanced tracking reliability by adding surface features to point features and deploying YOLOv4-tiny for dynamic region detection. MVS-SLAM [38] combined YOLOv7-provided semantic priors with K -means clustering for dynamic object rejection. DGM-VINS [39] introduced a geometric dynamic feature extraction module integrated with multiple constraints for dynamic feature detection. Dynam-SLAM [25] integrated stereo scene flow and IMU data for dynamic feature detection, merging these with static features and IMU measurements for nonlinear optimization, yielding promising results. Recent work by Wen et al. [40] proposes a method that combines semantic segmentation information and spatial motion information to handle dynamic objects in outdoor scenes, improving positioning accuracy in sequences with many dynamic objects. Another recent study introduces CD-SLAM [41], a real-time stereo visual–inertial SLAM system designed for complex dynamic environments, which enhances tracking by utilizing YOLOv5 for object detection and integrating IMU data for improved robustness.

This comprehensive overview highlights the evolving landscape of SLAM methodologies, showcasing the shift from geometry-based to semantic-integrated approaches, addressing the challenges posed by dynamic environments.

C. Summary

While the integration of line and plane features in SLAM, as demonstrated by [5] and [28], has improved stability in static scenes, these methods often lack semantic understanding, limiting their effectiveness in dynamic environments. Similarly, dynamic scene SLAM methods, such as DS-SLAM [18] and Dynam-SLAM [25], typically rely on single mechanisms (e.g., semantic segmentation or optical flow) to handle dynamic objects. These approaches can struggle with untrained dynamic objects not recognized by deep learning models or suffer from inaccuracies due to the quality of optical flow, especially in scenes with rapid camera motion or significant occlusions.

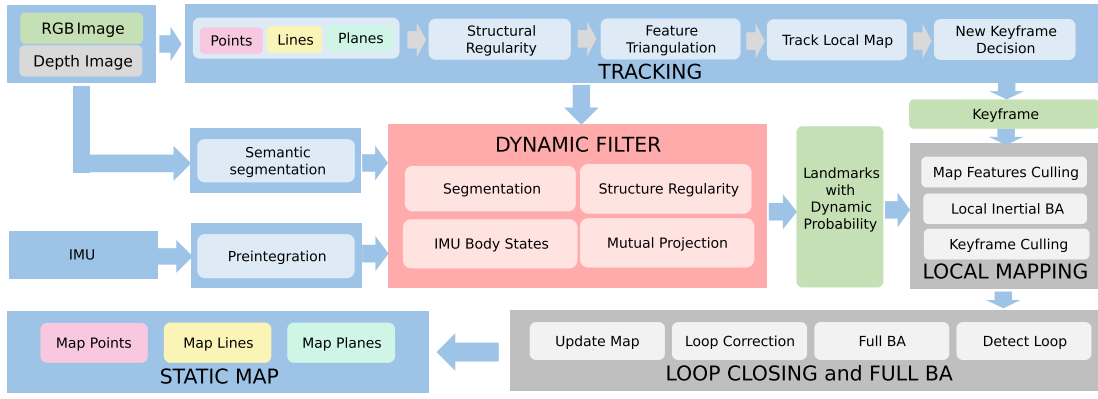


Fig. 2. Framework of the proposed MSSD-SLAM. The initial step extracts features from RGB-D frames, followed by the constraint of line and plane features based on structural regularity. The semantic segmentation is then performed in parallel, and the Dynamic Filter is utilized to process dynamic objects. Subsequently, the pose is computed and optimized. The system also includes loop closure detection and full BA optimization to ensure global consistency and accuracy. The final stage creates a static map consisting of points, lines, and planes.

Moreover, many existing dynamic SLAM solutions face challenges in weakly textured dynamic environments, where the removal of dynamic features leads to insufficient tracking features, causing instability or loss of tracking. This is a critical issue in environments with prevalent nontextured surfaces, such as indoor scenes dominated by plain walls.

Our MSSD-SLAM system addresses these limitations by harnessing a comprehensive blend of multiple features (points, lines, and planes), semantic information, structural constraints, and IMU data. This multifaceted approach reduces reliance on any single method for dynamic object detection, enhancing robustness and enabling stable operation even in feature-sparse dynamic scenes. The integration of IMU information allows our system to maintain accurate tracking despite rapid movements or the presence of untrained objects.

By combining these elements, our MSSD-SLAM system overcomes the limitations of previous methods and demonstrates superior performance in a wider range of dynamic environments, representing a significant advancement in the field.

III. SYSTEM OVERVIEW

The system's overall framework is illustrated in Fig. 2. It accepts RGB images, depth images, and IMU measurements as inputs. Initially, the system extracts point, line, and plane features from RGB and depth images and proceeds to triangulate them. Using line features and plane normals, it estimates environmental structure information, specifically the MWF. Line and plane features are constrained to simplify their representation based on structural regularity. Subsequently, IMU measurements are employed to estimate the IMU body state through preintegration. Preintegration of these measurements between successive image frames facilitates the determination of rotation, velocity, and position information. Concurrently, the semantic segmentation thread uses deep learning techniques to semantically segment images and feed results to the tracking thread.

The Dynamic Filter presented in this study utilizes semantic and structure regularity of the environment, as well as state information from IMU integration, to compute the dynamic probabilities of landmarks. These calculations utilize

short-term information between frames. In addition, the Dynamic Filter employs forward and backward projections among the landmarks of local keyframes to further discern the dynamic landmarks. During the BA optimization of local mapping, landmarks are assigned different weights according to their dynamic probabilities. The mapping thread utilizes static features to construct a static map containing points, lines, and planes.

To ensure long-term accuracy and consistency of the map, loop closure detection is performed. When a loop closure is detected, the system performs a full BA optimization to correct any accumulated drift by optimizing all keyframes and landmarks. This process refines the entire map and adjusts the camera poses to maintain global consistency.

Nomenclature provides a comprehensive list of symbols and their meanings used throughout this article.

IV. FEATURE REPRESENTATION AND STRUCTURAL REGULARITY

A. Feature Extraction and Representation

Point features are the primary feature type in visual SLAM. However, indoor scenes often have walls and floors with low-texture surfaces. In addition, removing dynamic features significantly reduces the number of available features, thus increasing the risk of tracking loss in the system. Indoor scenes typically exhibit multiple straight lines and planes. Therefore, we incorporate line and plane features to enhance the robustness and stability of the system.

1) *Points*: For features, we use ORB features and represent the coordinates of the feature in the image as $\mathbf{p}_c = (u, v)^T$ and the corresponding 3-D point as $\mathbf{P}_w = (X, Y, Z)^T$.

2) *Lines*: The 2-D line features are extracted and matched, following the same procedure as DRG-SLAM [26]. Spatial lines are represented using Plücker coordinates. Let $\mathbf{d} \in \mathbb{R}^3$ be the direction vector of the spatial line and $\mathbf{n} \in \mathbb{R}^3$ be the normal vector of the plane determined by the line and the origin of the coordinates. The line's Plücker coordinates can be expressed as $\mathbf{L} = (\mathbf{n}^T, \mathbf{d}^T)^T \in \mathbb{R}^6$. Suppose the observation of a certain spatial line $\mathbf{L}_w$ in an image is a 2-D line segment $\mathbf{l}_1$, and the plane determined by the camera center and the line segment is denoted as $\boldsymbol{\pi} = (\pi_x, \pi_y, \pi_z, \pi_w)^T$. For two

observed frames, the planes π_1 and π_2 can be determined by the intersection of these two planes to correspond to the spatial lines. The dual Plücker matrix is defined as follows:

$$\mathbf{L}^* = \begin{bmatrix} \mathbf{d}^\wedge & \mathbf{n} \\ -\mathbf{n}^\top & \mathbf{0} \end{bmatrix} = \pi_1 \pi_2^\top - \pi_2 \pi_1^\top \in \mathbb{R}^{4 \times 4}. \quad (1)$$

Then, the Plücker coordinates of the spatial line can be derived from the dual Plücker matrix.

Plücker coordinates represent the infinite lines in 3-D space. We employ endpoint trimming to determine the endpoints of the spatial lines for visualization. Consider the line $\mathbf{l}_1 = \mathbf{s}_1 \times \mathbf{e}_1 = (l_1, l_2, l_3)^\top$. Let $\mathbf{l}_s$ be the line perpendicular to $\mathbf{l}_1$. Define $\mathbf{s}_1 = (s_1, s_2, 1)^\top$. Another point on $\mathbf{l}_s$ can be obtained as $\mathbf{s}'_1 = (s_1 + l_1, s_2 + l_2, 1)$. Consequently, $\mathbf{l}_s$ can be defined as $\mathbf{s}_1 \times \mathbf{s}'_1$. The plane π_s can be obtained as $\pi_s = \mathbf{H}^\top \mathbf{l}_s$, with $\mathbf{H}$ representing the projection transformation matrix. The intersection point $\mathbf{S}$ of the plane π_s and the spatial line $\mathbf{L}$ can be determined as $\mathbf{S} = \mathbf{L}^* \pi_s$, where

$$\mathbf{L}^* = \begin{bmatrix} \mathbf{n}^\wedge & \mathbf{d} \\ -\mathbf{d}^\top & \mathbf{0} \end{bmatrix}. \quad (2)$$

The other endpoint, $\mathbf{E}$, can be obtained in the same way as $\mathbf{S}$.

3) *Planes*: The agglomerative hierarchical clustering method [42] was employed to extract plane features. The plane feature is represented by the Hesse normal form, denoted as $\pi = (\mathbf{m}^\top, d)^\top$, where $\mathbf{m}$ refers to the unit normal vector of the plane, while d denotes the plane distance from the origin. Since a plane has three degrees of freedom, the representation of these four parameters is considered overparameterized. The unit normal vector is subject to the $\|\mathbf{m}\| = 1$ constraint, limiting it to only two degrees of freedom. Therefore, it can be expressed using spherical coordinates: $\pi = (\sin \theta \cos \varphi, \sin \theta \sin \varphi, \cos \theta, d)^\top$. The polar angle, θ , and azimuthal angle, φ , determine its direction.

B. Structural Regularity and Measurement Model for Line and Plane Features

Indoor environments, characterized by their structured constraints, traditionally leverage the MWF for representing these regularities. Prior methods, focusing on 2-D imagery, struggle with dynamic scene complexities, as line features critical for MWF estimation are often obscured by moving objects. Addressing this, our approach innovates by extracting and constructing 3-D line and plane features from multiframe observations, thereby overcoming the limitations of single-frame analysis. Integrating structured constraints post 3-D construction, our method uniquely circumvents dynamic interference, ensuring enhanced geometric consistency and robust MWF estimation with just one principal direction alongside gravity based on the manifold-constrained mean-shift algorithm [43], identified through IMU initialization. This novel strategy not only refines feature representation and camera pose estimation in dynamic environments but also underscores the inadequacy of relying solely on 2-D data for structured constraint application in dynamic SLAM contexts.

Structured lines are identified by calculating the angles between spatial lines and each of the three principal directions of the MWF separately to determine their directional

coherence. The world coordinate system is initially aligned with the MWF in the initialization phase. Let $\mathbf{R}_{cm}$ represent the detected MWF in the current frame, and $\mathbf{R}_{wc}$ denote the rotation matrix of the current frame in the world coordinate system. Then, the MWF in the world coordinate system, $\mathbf{R}_{wm}$, can be expressed as $\mathbf{R}_{wm} = \mathbf{R}_{wc} \mathbf{R}_{cm} = (\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z)$, where $\mathbf{e}_x = (1, 0, 0)^\top$, $\mathbf{e}_y = (0, 1, 0)^\top$, and $\mathbf{e}_z = (0, 0, 1)^\top$ are the three principal directions of the MWF. We compute the angle between the direction vector $\mathbf{d}$ of line $\mathbf{L}$ and the corresponding principal direction of the MWF. If this angle is below a specified threshold, the line is classified as a structured line in that direction. Structured lines are denoted as $\mathbf{l}_i = ((\mathbf{S} \times \mathbf{e}_i)^\top, \mathbf{e}_i^\top)^\top \in \mathbb{R}^6$, where $i = x, y$, and z represent the structured lines in different principal directions, and $\mathbf{S}$ represents the endpoint of the spatial line. Unstructured lines still use the original Plücker coordinates, ensuring a consistent parameterization for both structured and unstructured lines.

The spatial line can be transformed from the world coordinate system to the camera coordinate system using the rotation matrix $\mathbf{R}_{cw}$ and the translation vector $\mathbf{t}_{cw}$ as follows:

$$\mathbf{L}_c = \begin{bmatrix} \mathbf{n}_c \\ \mathbf{d}_c \end{bmatrix} = \mathcal{T}_{cw} \mathbf{L}_w = \begin{bmatrix} \mathbf{R}_{cw} & \mathbf{t}_{cw}^\top \mathbf{R}_{cw} \\ \mathbf{0} & \mathbf{R}_{cw} \end{bmatrix} \mathbf{L}_w. \quad (3)$$

Then, the line $\mathbf{L}_c$ can be projected onto the image plane by the following equation:

$$\mathbf{l}_c = \begin{bmatrix} l_1 \\ l_2 \\ l_3 \end{bmatrix} = \begin{bmatrix} f_y & 0 & 0 \\ 0 & f_x & 0 \\ -f_y c_x & -f_x c_y & f_x f_y \end{bmatrix} \mathbf{n}_c. \quad (4)$$

Here, f_x , f_y , c_x , and c_y are the camera's intrinsic parameters. Let $\mathbf{s}'$ and $\mathbf{e}'$ denote the endpoints of the observed line $\mathbf{l}'_c$ corresponding to the spatial line $\mathbf{L}_c$. The error function of the line is

$$\mathbf{r}_l = \begin{bmatrix} d(\mathbf{s}', \mathbf{l}_c) \\ d(\mathbf{e}', \mathbf{l}_c) \end{bmatrix} = \frac{1}{\sqrt{l_1^2 + l_2^2}} \begin{bmatrix} \mathbf{s}'^\top \mathbf{l}_c \\ \mathbf{e}'^\top \mathbf{l}_c \end{bmatrix}. \quad (5)$$

After detecting MWF, structured constraints can also be incorporated for planar features. For a planar feature represented by $\pi = (\mathbf{m}^\top, d)^\top$, the angle between the unit normal vector $\mathbf{m}$ and the dominant directions of MWF, namely, $\mathbf{e}_x$, $\mathbf{e}_y$, and $\mathbf{e}_z$, is calculated. If the angle is below a certain threshold, the plane is classified as a structured plane in that direction. The structured plane can be denoted as $\pi_i = (\mathbf{m}_i^\top, d_i)^\top = (\mathbf{e}_i^\top, -\mathbf{e}_i^\top \mathbf{x}_0)^\top$, where $i = x, y$, and z , and $\mathbf{x}_0$ represents the center of the plane. Unstructured planes continue to utilize the original coordinates. The map plane, denoted as π_w , when transformed to the current frame's coordinate system, becomes $\pi_c = \mathbf{T}_{cw}^\top \pi_w$, and the corresponding observed plane in the current frame is referred to as π'_c . The equation representing the plane error is

$$\mathbf{r}_\pi = s(\pi_c) - s(\pi'_c). \quad (6)$$

Then, the overall optimization cost function of the system is

$$\mathbf{T}_{wi}^* = \arg \min_{\mathbf{R}_{wi}, \mathbf{t}_{wi}} \sum_{j \in \mathcal{P}} \rho_h \left(\mathbf{r}_{p_{ij}}^\top \Sigma_{p_{ij}}^{-1} \mathbf{r}_{p_{ij}} \right) + \sum_{k \in \mathcal{L}} \rho_h \left(\mathbf{r}_{l_{ik}}^\top \Sigma_{l_{ik}}^{-1} \mathbf{r}_{l_{ik}} \right) + \sum_{l \in \pi} \rho_h \left(\mathbf{r}_{p_{il}}^\top \Sigma_{p_{il}}^{-1} \mathbf{r}_{p_{il}} \right). \quad (7)$$

V. DYNAMIC FILTER

The presence of dynamic objects, such as pedestrians, in the real environment can lead to drift of the SLAM system. Thus, it is imperative to process dynamic objects to mitigate their impact. In our previous work, DRG-SLAM, we eliminate dynamic feature points directly by assessing their dynamism in each frame. However, this visual SLAM method remains susceptible to interference in highly dynamic scenes and heavily relies on semantic segmentation. Nevertheless, it may not adequately identify dynamic objects beyond semantic segmentation's capabilities. Moreover, the method relies solely on interframe information for dynamic object determination, overlooking longer term information. Hence, our proposal, the Dynamic Filter, incorporates semantic segmentation, environmental structural constraints, and IMU preintegration information to compute dynamic probabilities of features. The algorithm computes long-term dynamic object probabilities by projecting local keyframes onto one another. The derived dynamic probabilities from multiple sources are merged to form the dynamic probabilities associated with the corresponding landmarks. Any landmark with a dynamic probability above the threshold is flagged as an outlier, whereas those falling below the threshold are retained for pose estimation and mapping. The dynamic probabilities of the retained landmarks are kept up to date within the filter.

A. Compute Dynamic Probability by Semantic Segment

Our proposed method employs semantic segmentation in a dedicated thread to detect objects in the environment for dynamic object detection. Semantic segmentation is performed using the DeepLab v3 [44] deep learning network, which offers notable benefits in terms of accuracy and efficiency and can be executed in real time on personal laptops. The segmentation model is trained on the Pascal VOC dataset [45] and is capable of identifying 20 objects, including people and chairs.

To determine the motion direction and magnitude of corresponding feature points, we calculate the optical flow between the previous and current frames following the acquisition of the semantic segmentation results. The motion of feature points corresponding to moving objects in the environment is distinguishable from that of static feature points, facilitating the identification of abnormal dynamic features.

Let $\mathbf{p}_j = (u_j, v_j)^T$ denote a feature point in the current frame and a feature point in the previous frame as $\mathbf{p}_i = (u_i, v_i)^T$. The motion velocity of the feature point between the two frames is $\mathbf{v}_i = (\mathbf{p}_j - \mathbf{p}_i)^T = (u_j - u_i, v_j - v_i)^T$. The set of all feature points in the current frame is represented as $\mathcal{A}$, with a total number of feature points denoted as N . Given prior knowledge, we assume that people in the environment are moving. The semantic mask of *person* corresponds to the feature points that are considered dynamic points, denoted as $\mathcal{B}$. This feature point set contains M points. The feature points outside the mask are assumed to be static. Their average velocity, representing the static velocity, is computed

as follows:

$$\bar{\mathbf{v}}_s = \sum_{i \in \mathcal{B}} \mathbf{v}_i / (N - M). \quad (8)$$

Next, we calculate the velocity error of each feature point with respect to the static velocity by $\Delta \mathbf{v}_i = \mathbf{v}_i - \bar{\mathbf{v}}_s = (\delta u, \delta v)^T$. The distance error is represented as $d_i = \|\Delta \mathbf{v}_i\|$ and the angle error as $\delta \theta_i = \text{atan2}(\delta v, \delta u)$. The sigmoid function is used to convert these errors into dynamic probabilities

$$P_{\text{seg}} = \frac{1}{1 + e^{-(\alpha(d_i - d_{th}) + (1 - \alpha)(\delta \theta - \theta_{th}))}}. \quad (9)$$

Here, α denotes the weight for the distance error, while d_{th} and θ_{th} are threshold values for the distance and angle, respectively. These threshold values adjust the translation of the sigmoid function on the x -axis to accurately depict dynamic probabilities.

B. Compute Dynamic Probability by Structural Regularity

There may be additional dynamic objects in the scene that are not detected by the network due to the limited types of objects that can be detected by semantic segmentation. Therefore, additional dynamic object processing is required.

The MWF in the environment remains unaffected by camera motion and dynamic objects, making it applicable for determining dynamic objects. Let $\mathbf{R}_{wm}$ represent the MWF in the world coordinate system and $\mathbf{R}_{cm}$ be the MWF detected in the current frame. The rotation of the current frame in the world coordinate system can be obtained as $\mathbf{R}_{wc} = \mathbf{R}_{wm} \mathbf{R}_{cm}^T$. By optimizing the translation only, we can estimate the translation of the current frame in the world coordinate system as $\mathbf{t}_{wc}$. Thus, the rotation from the previous frame to the current frame can be calculated as $\mathbf{R}_{ji} = \mathbf{R}_{jm} \mathbf{R}_{im}^T$ and the translation as $\mathbf{t}_{ji} = \mathbf{t}_{cw} - \mathbf{R}_{ji} \mathbf{t}_{iw}$. Utilizing the camera intrinsic matrix $\mathbf{K}$, we can get the fundamental matrix $\mathbf{F} = \mathbf{K}^{-T} \mathbf{t}_{ji} \wedge \mathbf{R}_{ji} \mathbf{K}^{-1}$. Let $\mathbf{p}_j$ represent the current frame's feature point, while $\mathbf{p}_i$ is its corresponding matched feature point in the previous frame. The epipolar line that corresponds to $\mathbf{p}_j$ is defined as $\mathbf{l}_j = \mathbf{F}^T \mathbf{p}_i = (l_1, l_2, l_3)^T$. The epipolar constraint $\mathbf{p}_j^T \mathbf{F} \mathbf{p}_i = 0$ holds for static feature points. However, dynamic points deviate from this constraint, and the distance from the feature point to the epipolar line can be used to measure the dynamic status of the point. Let $d_j = |\mathbf{p}_j^T \mathbf{l}_j| / (l_1^2 + l_2^2)^{1/2}$ be the distance from point $\mathbf{p}_j$ to the epipolar line $\mathbf{l}_j$. The dynamic probability can be computed by

$$P_{\text{MWF}} = \frac{1}{1 + e^{-(\alpha(d_j - d_{th}))}}. \quad (10)$$

Here, α represents the weight for the distance error, and d_{th} is the threshold for the distance.

C. Compute Dynamic Probability by IMU Measurement

After the aforementioned steps, it is possible to identify most of the dynamic features. However, there may be circumstances in which MWF is absent in the environment or situations where MWF cannot be detected due to obstructions caused by moving objects in highly dynamic conditions.

Consequently, relying solely on semantic segmentation and structured constraints proves inadequate, especially in the case of high-dynamic scenes. The IMU, being an internal sensor, remains unaffected by external dynamic objects. Therefore, the IMU is suitable for processing dynamic scenes.

The preintegration of IMU measurements [46] between two consecutive image frames i and j is performed to derive the transformation involving the following steps:

$$\begin{aligned}\Delta \mathbf{R}_{ij} &= \prod_{k=i}^{j-1} \text{Exp}\left(\left(\tilde{\boldsymbol{\omega}}_k - \mathbf{b}_k^g - \boldsymbol{\eta}_k^{gd}\right) \Delta t\right) \\ \Delta \mathbf{v}_{ij} &= \sum_{k=i}^{j-1} \Delta \mathbf{R}_{ik} \left(\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}\right) \Delta t \\ \Delta \mathbf{p}_{ij} &= \sum_{k=i}^{j-1} \left[\Delta \mathbf{v}_{ik} \Delta t + \frac{1}{2} \Delta \mathbf{R}_{ik} \left(\tilde{\mathbf{a}}_k - \mathbf{b}_k^a - \boldsymbol{\eta}_k^{ad}\right) \Delta t^2 \right] \quad (11)\end{aligned}$$

where $\mathbf{R}$ is rotation, $\mathbf{t}$ is translation, $\mathbf{v}$ is linear velocity, $\boldsymbol{\omega}$ is angular velocity, $\mathbf{a}$ is acceleration, $\tilde{\boldsymbol{\omega}}$ and $\tilde{\mathbf{a}}$ are the measurements of angular velocity and acceleration, respectively, $\mathbf{g}$ is the gravity vector, $\mathbf{b}^g$ and $\mathbf{b}^a$ are the zero biases of the gyroscope and accelerometer, respectively, and $\boldsymbol{\eta}^{gd}$ and $\boldsymbol{\eta}^{ad}$ are the discretized time measurement noises of the gyroscope and accelerometer, respectively.

According to the preintegration quantity of the IMU, we can obtain the rotation, velocity, and translation estimated by the IMU at time j as follows:

$$\begin{aligned}\mathbf{R}_j &= \mathbf{R}_i \Delta \mathbf{R}_{ij}, \\ \mathbf{v}_j &= \mathbf{v}_i + \mathbf{g} \Delta t_{ij} + \mathbf{R}_i \Delta \mathbf{v}_{ij}, \\ \mathbf{p}_j &= \mathbf{p}_i + \mathbf{v}_i \Delta t_{ij} + \frac{1}{2} \mathbf{g} \Delta t_{ij}^2 + \frac{1}{2} \mathbf{R}_i \Delta \mathbf{p}_{ij}.\end{aligned} \quad (12)$$

The above variables are represented in the world coordinate system, namely, $\mathbf{R}_j = \mathbf{R}_{wb_j}$, $\mathbf{v}_j = \mathbf{v}_{wb_j}$, and $\mathbf{t}_j = \mathbf{t}_{wb_j}$. Let the current IMU pose transformation matrix be $\mathbf{T}_{wb_j}$. Then, the pose of the current frame in the camera coordinate system can be represented as: $\mathbf{T}_{wc_j} = \mathbf{T}_{wb_j} \mathbf{T}_{bc}$, where $\mathbf{T}_{bc}$ represents the extrinsic matrix between the IMU and the camera. Hence, the pose estimated by the IMU from the previous frame to the current frame can be determined as $\mathbf{T}_{c_j c_i} = \mathbf{T}_{wc_j}^{-1} \mathbf{T}_{wc_i}$.

Let $\mathbf{P}_i$ denote a map point, and its projection in the previous and current frame images as key points $\mathbf{p}_{c_i}$ and $\mathbf{p}_{c_j}$, respectively. The reprojection error of the point can be computed through $r_i = \|\mathbf{p}_{c_j} - \mathbf{K} \mathbf{T}_{c_j c_i} \mathbf{P}_i\|$. As the pose estimated by the IMU remains unaffected by dynamic objects, the reprojection error r_i for map points associated with dynamic objects will exceed the expected range. Similarly, the error can be converted into dynamic probability by the sigmoid function

$$P_{\text{IMU}} = \frac{1}{1 + e^{-\alpha(r_i - r_{th})}}. \quad (13)$$

D. Compute Dynamic Probability by Projection

The previously discussed method relies solely on short-term information, utilizing only the data between two frames to detect dynamic objects. The landmarks of objects that are temporarily stationary but subsequently move are retained in

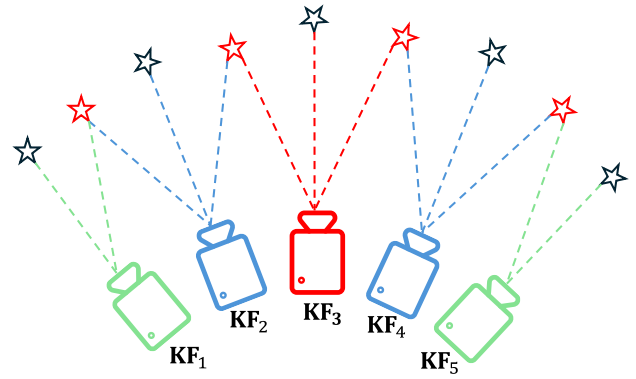


Fig. 3. Schematic illustrating covisible keyframe relationships. $\mathbf{KF}_2$ and $\mathbf{KF}_4$ are the primary covisible keyframe of current keyframe $\mathbf{KF}_3$, and $\mathbf{KF}_1$ and $\mathbf{KF}_5$ are the secondary covisible keyframes.

the map. To enhance the detection of such dynamic objects, we leverage the information among multiple keyframes. Keyframes that observe the same map points are referred to as covisible keyframes. As illustrated in Fig. 3, the current keyframe ($\mathbf{KF}_3$) observes the same map points as the primary covisible keyframes ($\mathbf{KF}_2$, $\mathbf{KF}_4$). The secondary covisible keyframes ($\mathbf{KF}_1$, $\mathbf{KF}_5$) observe the same map points as the primary covisible keyframes. Now, we take the top- m primary covisible keyframes with the highest covisibility with the current keyframe and then select the top- n secondary covisible keyframes with the highest covisibility with the primary covisible keyframes of current frame.

Let $\mathcal{K}_n$ represent the set of all covisible keyframes of the current keyframe, and $\mathcal{P}_n$ denotes the set of map points observable by these keyframes. First, forward projection is conducted by projecting the map points of the current keyframe onto the primary and secondary covisible keyframes to identify dynamic points. We designate the current keyframe as $\mathbf{KF}_i$ and the other keyframes within the covisible keyframes as $\mathbf{KF}_j$. Assuming that the pose transformation from $\mathbf{KF}_i$ to $\mathbf{KF}_j$ is $\mathbf{T}_{ji}$, the map point $\mathbf{P}_i \in \mathbb{R}^3$ in the current keyframe $\mathbf{KF}_i$ is projected onto $\mathbf{KF}_j$, resulting in $\mathbf{P}_i^j = \mathbf{T}_{ji} \mathbf{P}_i$. Subsequently, $\mathbf{P}_i^j$ is projected onto the image of $\mathbf{KF}_j$ to attain the keypoint $\mathbf{p}_i^j \in \mathbb{R}^2$. We search for a matching feature point $\mathbf{p}_j$ in the feature points of $\mathbf{KF}_j$ and then obtain the corresponding map point $\mathbf{P}_j \in \mathbb{R}^3$. In a static environment, the distance between $\mathbf{P}_i^j$ and $\mathbf{P}_j$ should be small; otherwise, the map point might correspond to a dynamic object. The distance error of the map point is defined as $d_{ji} = \|\mathbf{P}_i^j - \mathbf{P}_j\|$. The average distance error of the map point projected onto all covisible keyframes is calculated as follows: $\bar{d}_{ji} = (1/N) \sum_{j \in \mathcal{K}_n} \|\mathbf{P}_i^j - \mathbf{P}_j\|$.

In addition, we perform backward projection by projecting the map points of the first-level and second-level covisible keyframes onto the current keyframe. This method is similar to the forward projection. The projection of map point $\mathbf{P}_j$ from the covisible keyframe to the current keyframe is denoted as $\mathbf{P}_j^i$. The average distance error is then calculated as follows: $\bar{d}_{ij} = (1/N) \sum_{j \in \mathcal{K}_n} \|\mathbf{P}_j^i - \mathbf{P}_i\|$.

Finally, we convert it to a dynamic probability by

$$P_{\text{proj}} = \frac{1}{1 + e^{-\alpha((\bar{d}_{ij} + \bar{d}_{ji})/2 - d_{th})}}. \quad (14)$$

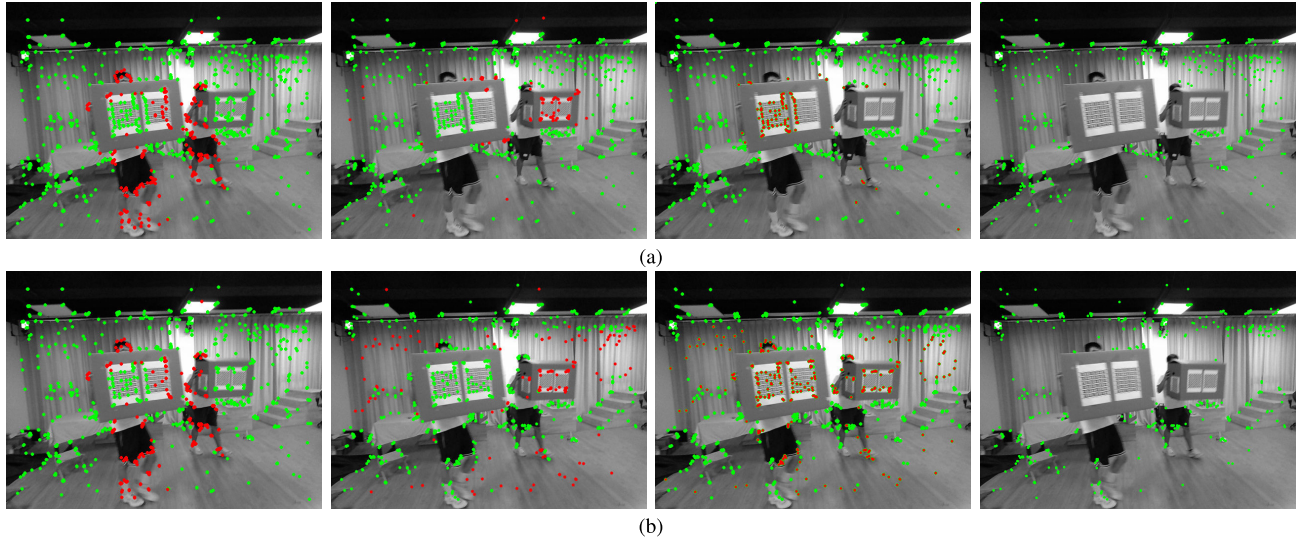


Fig. 4. Dynamic features detected by the Dynamic Filter; red points indicate feature points with dynamic probability greater than the threshold value, while other feature points are colored green. (a) Dynamic features detected by the progressive update policy. (b) Dynamic features detected by the comprehensive update policy. Each column of (a) and (b), from left to right, presents the dynamic features computed by semantic segmentation, structured constraints, and IMU observations, respectively. The last column displays the ultimate results for static points.

E. Dynamic Probabilities Fusion

We have obtained dynamic probabilities for semantic segmentation (P_{seg}), structured constraint (P_{MWF}), IMU pose estimation (P_{IMU}), and forward/backward projection (P_{proj}). The dynamic probability in each observation can be updated using the binary Bayes filters algorithm. Assuming that the state of the map points (dynamic or static) remains constant during observations, the confidence $\text{bel}_t(x)$ is solely determined by the measurements: $P(x | z_{1:t}, u_{1:t}) = P(x | z_{1:t})$. The state x can be either static or dynamic. The log-odds function ($L(x)$) is defined as $L(x) = \log(p(x)/(1 - p(x)))$. Next, the update is performed using the equation

$$L_t = L_{t-1} + \log \frac{P(x | z_t)}{1 - P(x | z_t)} - \log \frac{P(x)}{1 - P(x)}. \quad (15)$$

Here, $P(x | z_t)$ represents the inverse measurement model, and in our implementation, it corresponds to the dynamic probability obtained through above methods. $P(x)$ denotes the prior probability of state x . Initially, we set $P(x) = 0.5$ to reflect an equal likelihood of the map points being dynamic or static. Then, the dynamic probability can be recovered from the log-odds ratio by the following equation:

$$P_{\text{final}} = 1 - \frac{1}{1 + e^{(L_t)}}. \quad (16)$$

As mentioned above, the feature points in the current frame are categorized as static or dynamic through semantic segmentation and feature point motion analysis. The dynamic probability is then updated based on structured constraints and IMU observations. Two distinct policies are employed to update the dynamic probability. The first policy, referred to as the progressive update policy, updates only the static features calculated from the previous observation. If any feature points yield a dynamic probability above the threshold, they are categorized into the dynamic point set and are not involved in further processing. However, those falling below the threshold are allocated to the static feature set. The second policy, known

as the comprehensive update policy, updates the dynamic probabilities of all features in each observation. Feature points with dynamic probabilities surpassing a designated threshold are included in the dynamic point set.

Let us consider three observations, denoted as A , B , and C . We determine their confidences, essentially the probability of the accuracy of these observations, as P_A , P_B , and P_C , and their complements are represented as $\bar{P}_A$, $\bar{P}_B$, and $\bar{P}_C$. The accuracy of the judgment of dynamic feature points varies with each update. If the feature is updated i times, its accuracy is denoted as $P(D | Z_i)$. In the progressive update policy, the probability of a dynamic point being updated is denoted as follows:

$$P_{\text{update1}} = P_{A+B+C} = P_A + P_B \cdot \bar{P}_A + P_C \cdot \bar{P}_A \cdot \bar{P}_B. \quad (17)$$

Since each dynamic point is updated at most once, the probability of correctly estimating the dynamic point is $P_{D1} = P_{\text{update1}} \cdot P_{D|Z1}$.

For the comprehensive update policy, where all features are updated each time, the probabilities that a feature being updated once, twice, and three times are

$$\begin{aligned} P_{Z1} &= P_A \cdot \bar{P}_B \cdot \bar{P}_C + \bar{P}_A \cdot P_B \cdot \bar{P}_C + \bar{P}_A \cdot \bar{P}_B \cdot P_C \\ P_{Z2} &= P_A \cdot P_B \cdot \bar{P}_C + P_A \cdot \bar{P}_B \cdot P_C + \bar{P}_A \cdot P_B \cdot P_C \\ P_{Z3} &= P_A \cdot P_B \cdot P_C. \end{aligned} \quad (18)$$

Then, the confidence level that the dynamic feature points are correctly judged is

$$P_{D2} = \sum_{i=0}^3 P_{Zi} \cdot P_{D|Zi}. \quad (19)$$

The confidence difference between the two strategies is

$$\begin{aligned} \Delta P &= P_{D2} - P_{D1} \\ &= (P_{D|Z2} - P_{D|Z1})P_{Z2} + (P_{D|Z3} - P_{D|Z1})P_{Z3}. \end{aligned} \quad (20)$$

Since accuracy generally increases with the number of observations, it can be inferred that the first policy is more accurate

Algorithm 1 Dynamic Filter Algorithm

Require: Current Frame F_i , Last Frame F_{i-1} , Covisible KeyFrames KFs , All features set $\mathcal{A}(t)$, Dynamic features set $\mathcal{D}(t)$, Static features set $\mathcal{S}(t)$, Observation Z_{seg} , Z_{MWF} , Z_{IMU}

- 1: $P(x_0) = 0.5$, $L_0(x) = \log \frac{P(x_0)}{1-P(x_0)}$, $\mathcal{S}(0) = \mathcal{A}(t)$, $\mathcal{D}(0) = \emptyset$
- 2: **for** all features $\mathbf{p}_i$ in $\mathcal{S}(t)$ **do**
- 3: **for** all observation z_m **do**
- 4: $P(x|z_m) \leftarrow$ Compute dynamic probability $P(t)_{Z_m}$
- 5: $L_t = L_{t-1} + \log \frac{P(x|z_m)}{1-P(x|z_m)} - L_0(x)$
- 6: $P(t)_{final} = 1 - \frac{1}{1+\exp(L_t)}$
- 7: $L_{t-1} = L_t$
- 8: **if** $P(t)_{final} < P_{th}$ **then**
- 9: put $\mathbf{p}_i$ in $\mathcal{S}(t)$
- 10: **else**
- 11: remove $\mathbf{p}_i$ from $\mathcal{S}$
- 12: put $\mathbf{p}_i$ in $\mathcal{D}(t)$
- 13: **end if**
- 14: **end for**
- 15: **if** $P(t)_{final} < P_{th}$ **then**
- 16: dynamic probability of landmark $\mathbf{P}_i \leftarrow P(t)_{final}$
- 17: **else**
- 18: set landmark $\mathbf{P}_i$ as dynamic
- 19: **end if**
- 20: **end for**
- 21: **for** all landmarks $\mathbf{P}_i$ in Current Frame F_i **do**
- 22: **for** all covisible keyframe KFs **do**
- 23: $P(t)_{proj} \leftarrow$ forward/backward projection
- 24: **end for**
- 25: **if** $P(t)_{final} < P_{th}$ **then**
- 26: dynamic probability of landmark $\mathbf{P}_i \leftarrow P(t)_{proj}$
- 27: **else**
- 28: set landmark $\mathbf{P}_i$ as dynamic
- 29: **end if**
- 30: **end for**

than the second policy, with $P_{D|Z_3} > P_{D|Z_2} > P_{D|Z_1}$. Nevertheless, the first policy is more efficient than the second policy, since it only updates static features.

Fig. 4 illustrates the results of both policies concerning the computation of dynamic feature points. Dynamic feature points exceeding the threshold are represented in red, while those below the threshold are depicted in green. While both methods can identify dynamic points, the progressive update policy achieves higher precision by utilizing filtered static points. In contrast, the comprehensive update policy estimates all feature points and is, therefore, more versatile. Nevertheless, it is susceptible to overfitting, where some static points are misclassified as dynamic points, thereby reducing the stability of the SLAM system. This issue is particularly significant in scenarios with limited features.

Table I presents the time consumed by the two policies. The first policy has an average time consumption of 12.8 ms, whereas the second policy averages 16.3 ms. This validates the efficiency of the progressive policy. However, while the comprehensive policy is adept at excluding dynamic fea-

TABLE I
AVERAGE TIME COST OF DIFFERENT POLICIES

Method	Semantic Segment (ms)	Structural Regularity (ms)	IMU Measurement (ms)	Total Time (ms)
Progressive Policy	4.5	5.6	2.7	12.8
Comprehensive Policy	4.8	7.8	3.7	16.3

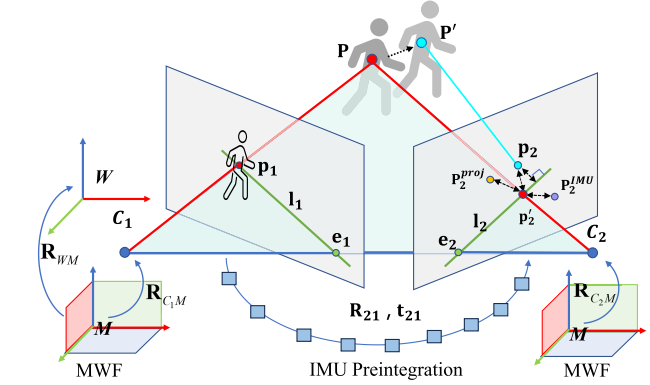


Fig. 5. Illustration of dynamic feature processing of Dynamic Filter.

ture influences, it is essential to prioritize stable tracking and real-time performance in SLAM systems. Therefore, the progressive update policy is preferable.

After completing all the aforementioned calculations, landmarks are considered outliers and excluded from further processing if their final dynamic probability exceeds the threshold value. Conversely, if the dynamic probability is below the threshold, we use the inverse as the optimization weight.

For line and plane features, we enhance our computation methods to robustly determine their dynamic status. For line features, the dynamic likelihood is computed using both their midpoint and angle variation, while for plane features, the centroid is employed. To mitigate misclassification due to temporary occlusions, we integrate semantic information to assess proximity to potentially dynamic objects and calculate dynamic probability using projections across multiple keyframes. This ensures consistent positioning before and after occlusions, resulting in a lower dynamic probability over long-term observations. If the dynamic probability of a line's midpoint or angle variation, or a plane's centroid exceeds a predefined threshold, the feature is classified as dynamic. Conversely, if the dynamic probability falls below this threshold, it is utilized as a weight in the back-end optimization process. This approach allows for a nuanced treatment of dynamic features, ensuring that their potential impact on the SLAM system's accuracy and stability is appropriately mitigated. The procedure for the Dynamic Filter is outlined in Algorithm 1.

Fig. 5 illustrates the judgment strategy of dynamic points by the Dynamic Filter. $\mathbf{P}$ represents a landmark, with its corresponding feature point denoted as $\mathbf{p}_1$ in the first frame. The theoretical matched point in the second frame, denoted as $\mathbf{p}_2'$, should lie on the epipolar line $\mathbf{l}_2$. When $\mathbf{P}$ moves to $\mathbf{P}'$ at the time of the second frame, the matched feature point becomes

TABLE II
COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR TUM RGB-D SEQUENCES¹

Sequences / Dynamic Level	MSSD-SLAM		MSSD-SLAM w/o PL		MSSD-SLAM w/o D		DRG-SLAM		DS-SLAM		Dynamic-VINS		SG-SLAM		ORB-SLAM3	
	Trans.(m)	Rot.(rad)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(m)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>sitting_half</i> / *	0.0354	0.3468	0.0621	0.3936	0.0572	0.3886	0.0734	0.4195	0.0151	0.3940	0.0791	0.7153	0.0268	0.4171	0.0190	0.3574
<i>sitting_rpy</i> / *	0.0236	0.3450	0.0291	0.4226	0.0275	0.4827	0.0317	0.4996	0.0287	0.4074	0.0983	0.4106	0.0244	0.4140	0.0313	0.4540
<i>sitting_static</i> / *	0.0066	0.1503	0.0119	0.3347	0.0073	0.1683	0.0072	0.1511	0.0116	0.3557	0.0110	0.1392	0.0128	0.1675	0.0094	0.1627
<i>sitting_xyz</i> / *	0.0101	0.3139	0.0191	0.4112	0.0148	0.3260	0.0136	0.3175	0.0309	0.5470	0.0328	0.3546	0.0196	0.3330	0.0311	0.3160
<i>walk_half</i> / ***	0.0173	0.3960	0.0269	0.4100	0.6044	0.5667	0.0258	0.7350	0.0315	0.4255	0.0585	0.4712	0.0309	0.4368	0.4271	0.5213
<i>walk_rpy</i> / ***	0.0328	0.4788	0.0404	0.5089	1.0960	1.3481	0.0424	0.8347	0.4462	0.5786	0.1667	0.5186	0.0332	0.5924	0.8290	0.6413
<i>walk_static</i> / ***	0.0179	0.1855	0.0187	0.1812	0.2882	0.2788	0.0111	0.2350	0.0070	0.1649	0.0493	0.7649	0.0080	0.1894	0.1435	0.3485
<i>walk_xyz</i> / ***	0.0136	0.3926	0.0147	0.3945	0.8077	0.5936	0.0217	0.5901	0.0323	0.5270	0.0501	0.3861	0.0174	0.4000	0.5009	0.5513
<i>Average</i>	0.0197	0.3261	0.0279	0.3821	0.3629	0.5191	0.0284	0.4728	0.0754	0.4250	0.0682	0.4701	0.0216	0.3688	0.2489	0.4191

¹ Abbreviations: MSSD-SLAM w/o PL —MSSD-SLAM without line and plane features, MSSD-SLAM w/o D —MSSD-SLAM without the proposed Dynamic Filter.

$\mathbf{p}_2$, and $\mathbf{p}_2^{\text{proj}}$ represents the projection of the local map point onto the feature point in that frame. $\mathbf{R}_{C_1M}$ and $\mathbf{R}_{C_2M}$ represent the rotations from MWF to the first and second frames, respectively. $\mathbf{R}_{WM}$ denotes the rotation from MWF to the world coordinate system. The poses between the two frames obtained through IMU preintegration are represented by $\mathbf{R}_{21}$ and $\mathbf{t}_{21}$. $\mathbf{p}_2^{\text{IMU}}$ represents the projected point calculated using preintegrated poses. These points exhibit errors in comparison with the theoretical point $\mathbf{p}_2'$, which are used to assess the dynamic probability. The detailed process has been previously elucidated.

VI. EXPERIMENTAL VALIDATION

Experimental verification was conducted on the TUM RGB-D dataset [47] and a self-recorded dataset, to validate the effectiveness of the proposed method. We also compared our method with other state-of-the-art open-source dynamic SLAM methods. All experiments were performed on a computer with an Intel Core i7-10750H CPU at 2.6 GHz and NVIDIA GeForce RTX 2060 GPU and 16-GB RAM.

A. TUM RGB-D Dataset

The TUM RGB-D dynamic dataset comprises RGB and depth images, primarily categorized into sitting and walking. In the sitting sequence, the scene is almost stationary, with two people sitting on chairs and only making some hand gestures, while the walking sequences capture the reciprocal movement of two persons in an office. Our method, based on ORB-SLAM3 [3], was compared with other advanced dynamic SLAM algorithms, including DS-SLAM [18], SG-SLAM [35], Dynamic-VINS [24], and our previous work, DRG-SLAM [26]. To validate the impacts of different modules, we conducted ablation experiments to individually assess the effects of removing structured features and the Dynamic Filter. As the dataset lacks IMU data, all algorithms operate in RGB-D mode.

For the translational component, we utilized the absolute pose error (APE) metric to analyze the global consistency of the SLAM trajectory. Misalignment between the reference coordinate systems of the SLAM trajectory and ground truth may result in alignment issues. Hence, the relative pose error (RPE) is utilized to evaluate the rotational error of the pose.

The root-mean-square error (RMSE) of APE and RPE is calculated using evo tool, and Nomenclature shows the results.

Stars indicate the dynamic levels of different sequences, with more stars representing a higher degree of dynamics. The results from Table II demonstrate that the proposed method outperforms other methods on the TUM RGB-D dynamic dataset. We achieved the average RMSE improvements of up to 92% compared with the original ORB-SLAM3. Given that the sitting sequence is almost stationary, each SLAM method achieved high localization accuracy. Our approach showed no significant difference even after the removal of the Dynamic Filter. SG-SLAM also showed good performance in the scene consisting solely of dynamic people. DS-SLAM shows competitive results in the *walking_static*, *walking_half*, and *walking_xyz* sequences, indicating robustness in moderate dynamic activities. However, in the *walking_rpy* sequence, where significant camera rotation occurs, DS-SLAM's performance drops due to its reliance on epipolar geometry constraints, which can be less effective under large rotations. In RGB-D mode, Dynamic-VINS is more susceptible to the influence of dynamic objects, particularly in the *walking_rpy* sequence, due to the absence of IMU data.

During the ablation study, the omission of the structural regularity module from our proposed SLAM system led to a marginal reduction in localization accuracy. This reduction underscores the module's role in bolstering robustness and enriching the mapping process. The exclusion of the Dynamic Filter module, on the other hand, resulted in notable trajectory deviation, emphasizing its crucial role in managing dynamic environments.

We undertook extensive ablation experiments to assess the Dynamic Filter further. Comprising semantic segmentation, structural regularity, IMU measurement, and projection modules, the Dynamic Filter is succinctly denoted as S, M, I, and P for convenience. Given the absence of IMU data in the TUM dataset, the IMU measurement module was excluded from consideration. Our ablation studies focused on the dynamic sequences of the TUM dataset, specifically the walking sequences, to evaluate effectiveness. Initial experiments involved deploying each module independently for dynamic object filtration, namely, MSSD-SLAM with S, MSSD-SLAM with M, and MSSD-SLAM with P, with findings documented in Table III. Utilizing single modules

TABLE III

COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR TUM RGB-D SEQUENCES OF ABLATION EXPERIMENTS¹

Sequences / Dynamic Level	MSSD-SLAM with S		MSSD-SLAM with M		MSSD-SLAM with P		MSSD-SLAM w/o S		MSSD-SLAM w/o M		MSSD-SLAM w/o P		MSSD-SLAM full	
	Trans.(m)	Rot.(rad)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(m)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>walk_half</i> / ***	0.1289	1.1204	0.1417	1.0986	0.1399	1.1006	0.0898	1.0043	0.0640	0.8102	0.0552	0.5874	0.0173	0.3960
<i>walk_rpy</i> / ***	0.1424	1.8885	0.1412	1.5363	0.1318	1.4947	0.0842	1.2257	0.0799	0.9720	0.0701	0.8018	0.0328	0.4788
<i>walk_static</i> / ***	0.0209	0.3385	0.0426	0.3134	0.0435	0.3604	0.0326	0.3235	0.0212	0.3121	0.0219	0.3252	0.0179	0.1855
<i>walk_xyz</i> / ***	0.0729	0.8147	0.2863	0.7420	0.8077	0.8585	0.1635	0.4458	0.0722	0.7261	0.0492	0.4143	0.0136	0.3926
<i>Average</i>	0.0913	1.0405	0.1529	0.9226	0.2807	0.9536	0.0925	0.7499	0.0593	0.7051	0.0491	0.5322	0.0204	0.3632

¹ MSSD-SLAM with S, MSSD-SLAM with M, and MSSD with P refer to the use of the Semantic Segmentation module, Structural Regularity module, and Projection module, respectively, in the proposed Dynamic Filter. Conversely, MSSD-SLAM w/o S, MSSD-SLAM w/o M, and MSSD w/o P indicate the removal of the Semantic Segmentation module, Structural Regularity module, and Projection module from Dynamic Filter in MSSD-SLAM, respectively.

resulted in reduced localization accuracy, particularly in highly dynamic environments. The semantic segmentation module, when used alone, exhibited the highest accuracy, highlighting its significant contribution to the Dynamic Filter. Further ablation experiments entailed removing these modules individually from the Dynamic Filter. The removal of the semantic segmentation module, thereby engaging the structural regularity and projection modules concurrently, led to a 39% and 67% accuracy improvement, respectively, compared with the solo use of the structural regularity or projection modules. Although the impact of removing structural regularity or projection was less pronounced, there was still a notable 35% and 46% accuracy enhancement over using the semantic segmentation module in isolation.

B. Self-Collected Dataset

To further validate the proposed algorithm, we conducted comprehensive tests using datasets recorded in real-world scenarios. These datasets were designed to challenge the robustness and versatility of our SLAM system in varying conditions.

1) *Dataset Description*: Our self-collected dataset consists of two parts: a handheld dataset and a mobile robot dataset. Both datasets were recorded in a room equipped with a Cortex motion capture system, providing sub-millimeter accuracy for ground-truth camera trajectories.¹ The room covers an area of approximately 36 m² (6 × 6 m). The handheld dataset was captured using a RealSense D455 camera held by a person, while the mobile robot dataset was captured using a mobile robot equipped with the camera (Fig. 6). Both datasets recorded RGB images, depth images, and IMU data. The sequences were categorized based on the level of dynamics.

1) *person_walk Sequences*: These sequences involve varying numbers of people walking back and forth in the scene. In the handheld dataset, *person_walk_1* features occasional movement, indicating low dynamics; *person_walk_2* has two people walking, representing moderate dynamics; and *person_walk_3* has three people moving continuously, creating high dynamics. In the mobile robot dataset, both *person_walk_1* and *person_walk_2* involve frequent movement, thus representing high-dynamic environments.

¹The dataset is available at <https://github.com/charleswangyanan/MSSD-SLAM-Dataset>

TABLE IV

AVERAGE DURATION AND TRAJECTORY LENGTH OF DATASETS

Dataset	Average Durations (s)	Average Length (m)
TUM	30.02	4.76
Handheld	52.21	18.04
Mobile Robot	47.59	23.63

- 2) *move_chair Sequence*: This sequence involves a person pushing a chair around the room. The camera often faces a white wall for extended periods, causing feature points to concentrate on the moving object, which increases the challenge for SLAM algorithms.
- 3) *hold_box_sequence*: In these sequences, people carry boxes that are not easily recognized by deep learning models, adding difficulty compared to sequences with only people. The level of dynamics varies. In moderate dynamic sequences, a single person carries a box around the scene, including *hold_box_1* and *hold_box_2* in the handheld dataset and *hold_box_1* in the mobile robot dataset. In high-dynamic sequences, two people carry a box together and places it on a table or each carry a box and walk around, such as *hold_box_3* and *hold_box_4* in the handheld dataset and *hold_box_2* and *hold_box_3* in the mobile robot dataset. Very high-dynamic sequences, including *hold_box_5* and *hold_box_6* in the handheld dataset, involve three people carrying boxes, occasionally blocking the camera by holding boxes in front of it or waving them.

Table IV compared the average trajectory lengths and durations of the TUM dynamic dataset with our self-collected dynamic datasets. The average duration and trajectory length for the four dynamic sequences in the TUM dataset are 30.02 s and 4.76 m, respectively. In contrast, our handheld dataset averages 52.21 s and 18.04 m, while the mobile robot dataset averages 47.59 s and 23.62 m. Both the duration and trajectory length of our datasets exceed those of the TUM dataset, demonstrating the robustness of our algorithm in handling longer sequences and larger areas.

2) *Results and Discussion*: We benchmarked our algorithm against several prominent SLAM methods, including ORB-SLAM3 [3], DS-SLAM [18], SG-SLAM [35], DynaVINS [17], Dynamic-VINS [24], and our previous work, DRG-SLAM [26]. Evaluations were conducted on a

TABLE V
COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR SELF-COLLECTED HANDHELD CAMERA SEQUENCES

Sequences / Dynamic Level	MSSD-SLAM		MSSD-SLAM w/o PL		MSSD-SLAM w/o D		DRG-SLAM		DS-SLAM		Dynamic-VINS		DynaVINS		SG-SLAM		ORB-SLAM3	
	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>person_walk_1</i> / *	0.0261	0.4334	0.0551	0.6849	0.0610	0.6966	0.0509	0.7106	0.0869	0.7821	0.1340	2.2631	0.1150	0.8607	0.0862	0.9956	0.0694	0.7437
<i>person_walk_2</i> / **	0.0891	0.5985	0.1013	0.9244	0.2091	1.2762	0.1306	1.0119	0.1188	0.6847	0.1427	1.5714	—	—	0.1411	1.2654	0.1872	1.2624
<i>person_walk_3</i> / ***	0.0959	0.4489	0.1471	0.5173	0.4486	0.5628	0.1794	1.3473	0.5333	2.2529	0.3500	2.7590	0.7504	4.0683	—	—	0.4683	1.1525
<i>move_chair</i> / ***	0.0972	0.4578	0.1329	0.8678	—	—	0.1477	1.0062	2.3624	1.1852	0.1488	0.7781	0.1807	0.7097	1.1000	1.3554	—	—
<i>hold_box_1</i> / **	0.1184	0.4267	0.1707	0.5698	0.1874	1.1893	0.3518	0.6143	0.2396	1.2754	0.3544	1.9774	0.4175	0.7959	—	—	—	—
<i>hold_box_2</i> / **	0.0762	0.4023	0.0988	0.4959	0.1663	1.1050	0.1149	0.5089	0.6074	1.0781	0.1971	0.9120	0.1336	0.6585	0.3646	0.7922	0.1905	1.1103
<i>hold_box_3</i> / ***	0.0841	0.5670	0.1072	0.6903	4.8551	0.7460	0.1949	1.0150	1.2445	1.0982	—	—	0.1123	1.4628	1.6494	0.7736	0.2876	0.8867
<i>hold_box_4</i> / ***	0.0692	0.2870	0.1051	0.3135	0.4658	0.3285	0.2064	0.7118	0.2956	0.7887	0.2319	1.6424	0.2388	1.3586	0.2629	0.7976	0.3114	0.7384
<i>hold_box_5</i> / ****	0.0807	0.6054	0.3335	0.8209	1.1410	0.7937	0.2510	0.6455	0.7765	1.0435	0.2120	1.2948	0.2083	1.2004	0.2138	1.2786	0.4034	0.8953
<i>hold_box_6</i> / ****	0.0769	0.4670	0.2829	0.8762	0.8826	0.8026	0.2469	0.9656	1.4767	1.0378	0.2469	1.9241	0.3341	1.7659	0.2599	1.1896	0.7633	0.8923
<i>Average</i>	0.0814	0.4794	0.1535	0.6761	0.9352	0.8334	0.1874	0.8537	0.7742	1.1227	0.2242	1.6803	0.2767	1.4312	0.5097	1.0560	0.3351	0.9602

TABLE VI
COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR SELF-COLLECTED MOBILE ROBOT SEQUENCES

Sequences / Dynamic Level	MSSD-SLAM		MSSD-SLAM w/o PL		MSSD-SLAM w/o D		DRG-SLAM		DS-SLAM		Dynamic-VINS		DynaVINS		SG-SLAM		ORB-SLAM3	
	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>person_stand</i> / static	0.0673	0.2677	0.1308	0.3223	—	—	0.1310	0.4337	0.1027	0.3242	0.0989	0.3953	0.2368	0.4629	0.1061	0.3653	0.1085	0.3241
<i>person_walk_1</i> / ***	0.0833	0.2727	0.1353	0.8085	0.4369	1.2691	0.1355	0.8091	0.2170	1.0652	0.1965	1.2503	0.3676	0.7269	0.1674	0.9462	0.4972	1.2194
<i>person_walk_2</i> / ****	0.1068	0.2563	0.1605	0.6579	0.6877	0.9917	0.1458	0.5467	0.1938	0.8150	0.1024	0.7110	1.2914	1.5994	0.1242	0.3616	0.7391	0.9611
<i>hold_box_1</i> / **	0.0892	0.2924	0.1537	0.6358	0.5835	1.0493	0.1566	0.6336	0.3687	1.0270	0.1494	0.8464	0.3728	1.0851	0.2526	1.0472	0.5833	1.6168
<i>hold_box_2</i> / ***	0.1036	0.4077	0.1915	0.3368	0.5047	0.3477	0.2386	0.9204	0.2406	1.0522	0.1331	0.7983	0.4390	1.8077	0.1510	0.5202	0.5963	1.0031
<i>hold_box_3</i> / ***	0.1117	0.2168	0.1716	0.5566	0.6512	0.9025	0.2087	0.4760	0.2878	0.6944	0.4517	1.3232	0.2019	0.6419	0.1623	0.5649	0.6359	0.9951
<i>Average</i>	0.0936	0.2856	0.1572	0.5530	0.5728	0.9121	0.1694	0.6366	0.2351	0.8297	0.1887	0.8874	0.4849	1.0540	0.1606	0.6342	0.5267	1.0199

TABLE VII
COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR SELF-COLLECTED HANDHELD CAMERA SEQUENCES OF ABLATION EXPERIMENTS

Sequences / Dynamic Level	MSSD-SLAM with S		MSSD-SLAM with M		MSSD-SLAM with I		MSSD-SLAM with P		MSSD-SLAM w/o S		MSSD-SLAM w/o M		MSSD-SLAM w/o I		MSSD-SLAM w/o P		MSSD-SLAM full	
	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>person_walk_2</i> / **	0.3147	1.2637	0.3423	1.2082	0.3652	1.2603	0.4199	1.2987	0.2603	1.0818	0.1680	1.0304	0.1566	1.0146	0.2119	1.0841	0.0891	0.5985
<i>person_walk_3</i> / ***	0.3366	1.6606	1.7092	1.6880	0.8966	1.6629	0.6417	1.6673	0.4032	1.5348	0.2391	0.5541	0.1785	1.5956	0.1819	0.5700	0.0959	0.4489
<i>move_chair</i> / ***	0.1904	0.8295	0.2506	0.9762	0.2979	1.2054	0.3206	1.4088	0.2267	1.4680	0.1659	0.7473	0.1583	0.6360	0.1620	0.5500	0.0972	0.4578
<i>hold_box_1</i> / **	0.2505	0.6030	0.2922	0.5022	0.2126	0.5417	0.2103	0.9743	0.1709	0.5164	0.1702	0.5053	0.1382	0.4836	0.1910	0.5046	0.1184	0.4267
<i>hold_box_2</i> / **	0.3325	0.7908	0.3250	0.8015	0.3511	0.9106	0.3996	0.9058	0.2604	0.8275	0.1686	0.6943	0.1467	0.6696	0.1489	0.6103	0.0762	0.4023
<i>hold_box_3</i> / ***	0.2331	0.8128	0.6675	0.8840	0.6937	0.9917	0.7251	0.9016	0.2743	0.7870	0.1345	0.7153	0.1250	0.6123	0.1533	0.6864	0.0841	0.5670
<i>hold_box_4</i> / ***	0.2178	0.7861	0.3230	0.7554	0.6369	0.9893	0.8710	0.9927	0.2972	0.9386	0.1653	0.6740	0.1448	0.6588	0.1304	0.5127	0.0692	0.2870
<i>hold_box_5</i> / ****	0.2988	0.9313	2.3955	1.7636	3.0044	1.8797	2.2354	1.8830	0.5354	0.9804	0.2505	0.9087	0.2222	0.8639	0.2084	0.8961	0.0807	0.6054
<i>hold_box_6</i> / ****	0.3376	0.7797	2.8554	1.9732	3.7468	1.9926	2.5243	1.8452	0.5849	0.6782	0.2758	0.6761	0.2167	0.5366	0.2495	0.6755	0.0769	0.4670
<i>Average</i>	0.2791	0.9397	1.0179	1.1725	1.1339	1.2705	0.9275	1.3197	0.3348	0.9792	0.1931	0.7228	0.1652	0.7857	0.1819	0.6766	0.0875	0.4734

¹ MSSD-SLAM with I and MSSD-SLAM w/o I refer to the use of the IMU Measurement module and the removal of the IMU Measurement module from the Dynamic Filter in MSSD-SLAM, respectively.

local computer using the respective deployed codes, with ORB-SLAM3 operating in RGB-D inertial mode. The results of the algorithm comparison for the handheld and mobile robot datasets are presented in Tables V and VI, respectively. The performance of each method will be discussed below.

a) *ORB-SLAM3*: ORB-SLAM3 generally performed well in sequences with minimal dynamics, such as *person_walk_1* and *person_walk_2* from the handheld dataset. However, it struggled significantly in highly dynamic sequences, such as *move_chair* and *hold_box_6*, exhibiting frequent tracking losses and substantial errors. The mobile robot sequences, particularly *hold_box_3*, further highlighted ORB-SLAM3's limitations in handling dynamic objects and complex motion, resulting in lower overall accuracy and stability.

b) *DS-SLAM*: DS-SLAM showed robust performance in moderately dynamic environments but encountered significant errors in high-dynamic sequences. In the handheld dataset,

it managed well in *person_walk_1* and *person_walk_2*, but struggled in *move_chair* and *hold_box_3* due to its reliance on epipolar geometry and semantic segmentation for dynamic object detection. This approach was insufficient in scenarios with unrecognized dynamic objects, such as moving boxes. Similarly, in the mobile robot dataset, DS-SLAM faced performance degradation in *hold_box* sequences due to its limited dynamic object handling capabilities.

c) *SG-SLAM*: SG-SLAM performed competitively in several sequences but showed vulnerability to dynamic disruptions due to its lack of IMU integration. While it maintained acceptable accuracy in low-dynamic scenarios of the handheld dataset, such as *person_walk_1* and *person_walk_2*, its performance deteriorated in high-dynamic sequences, such as *move_chair* and *hold_box_3*, where dynamic objects significantly impacted localization accuracy.

d) *Dynamic-VINS*: Dynamic-VINS demonstrated moderate performance across most sequences. Its integration of IMU

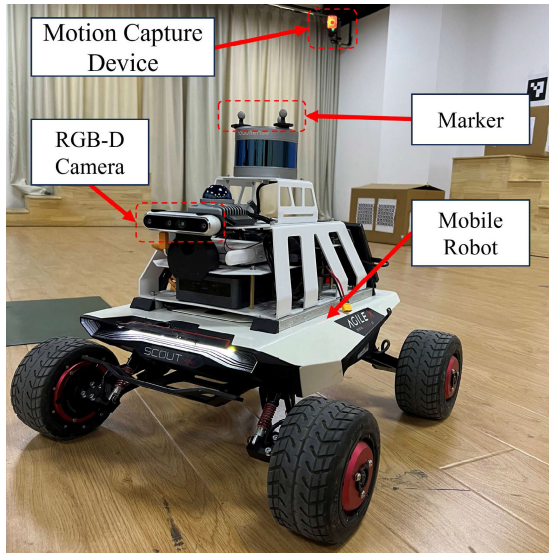


Fig. 6. Hardware configuration in dynamic scenario experiments with the mobile robot.

data helped mitigate drift, but it was still affected by dynamic objects, particularly in sequences with multiple moving people or objects. In the handheld dataset, Dynamic-VINS performed adequately in most sequences but struggled in *hold_box_3*. Similar challenges were observed in the *hold_box_3* in the mobile robot dataset, where dynamic elements and complex motion affected its overall performance.

e) *DynaVINS*: DynaVINS, which relies on monocular data and IMU integration, faced challenges in the tested sequences due to the lack of semantic segmentation and depth information. This resulted in lower accuracy and difficulty handling dynamic objects, particularly in *person_walk_2* and *person_walk_3* in the handheld dataset. Nevertheless, the algorithm demonstrated satisfactory performance in the mobile robot dataset, except for the *person_walk_2* sequence. As the TUM RGB-D dataset lacks IMU information, DynaVINS is only able to operate using the RGB images and, therefore, was unable to run successfully. Consequently, a comparison of it in the TUM dynamic dataset was not included.

f) *DRG-SLAM*: DRG-SLAM showed solid performance across various sequences, particularly in low-to-moderate dynamics. However, it still faced challenges in high-dynamic scenarios due to the lack of advanced dynamic object recognition and handling mechanisms present in MSSD-SLAM. In the handheld dataset, it performed well in *person_walk* sequences but encountered difficulties in *hold_box_5* and *hold_box_6*. Similarly, the method exhibits a slight increase in localization error in the *hold_box* sequence of the mobile robot dataset.

g) *MSSD-SLAM*: Our proposed MSSD-SLAM consistently outperformed other methods across most sequences. The integration of multifeature constraints, including point, line, and plane features, along with IMU data and advanced dynamic object recognition, allowed MSSD-SLAM to maintain high accuracy even in highly dynamic environments. The comprehensive use of structured features and the Dynamic Filter significantly enhanced its robustness and tracking performance. Notably, the average RMSE of MSSD-SLAM is

76% lower than ORB-SLAM3 in the handheld dataset and surpassed ORB-SLAM3 by an impressive 82% in the mobile robot dataset, cementing its status as a state-of-the-art solution.

Figs. 7 and 8 illustrate the trajectory analysis and corresponding error distributions for MSSD-SLAM and other methods in the *person_walk_1* sequence. MSSD-SLAM's integration of an efficient Dynamic Filter and line and plane features resulted in superior trajectory alignment with the ground-truth path, exhibiting lower errors and minimal outliers. This demonstrates MSSD-SLAM's adeptness in managing dynamic environments. Conversely, the other methods faced challenges with trajectory drift, exacerbated by the robot's low camera angle and the sparsity of feature points. The exclusion of dynamic points further intensified these problems, leading to broader error distributions and a spike in outliers. These findings underscore our method's superiority not only in trajectory precision but also in error consistency and predictability.

3) *Ablation Study*: Our ablation study focused on evaluating the contributions of individual components within MSSD-SLAM. The results indicated that removing line and plane features or the Dynamic Filter led to substantial performance degradation. According to Tables V and VI, the removal of line and plane features (MSSD-SLAM w/o PL) resulted in an average RMSE increase of 88% in the handheld dataset and 68% in the mobile robot dataset, highlighting the critical role these features play in maintaining robust performance.

The elimination of the Dynamic Filter (MSSD-SLAM w/o D) caused a significant increase in drift and tracking failures. The average RMSE exhibited a 11-fold increase in the handheld dataset and a sixfold increase in the mobile robot dataset, when compared with the original value. This highlights the necessity of the Dynamic Filter in handling dynamic objects and maintaining accuracy.

Further ablation experiments were conducted on the individual components of the Dynamic Filter, including semantic segmentation, structural regularity, IMU measurement, and projection modules, and the results are presented in Tables VII and VIII. The semantic segmentation module alone produced acceptable results, but the other modules, when used individually, led to larger errors and tracking losses in sequences, such as *hold_box_5* and *hold_box_6* in the handheld dataset. The structural regularity module alone was inadequate in maintaining tracking in the *person_walk_3* sequence in the handheld dataset, especially under heavy occlusions preventing the detection of Manhattan world coordinates. Further experiments entailed the sequential removal of each module from the Dynamic Filter. The elimination of the semantic segmentation module decreased accuracy. However, the accuracy showed the improvements of 9%, 32%, and 35%, respectively, in the handheld dataset and 21%, 37%, and 35% in the mobile robot dataset, respectively, compared with using the structural regularity, IMU measurement, and projection modules independently. The omission of the IMU measurement module impacted overall accuracy the least, while the removal of the structural regularity module had a significant effect, yet there was still about 31% improvement in accuracy in both handheld and mobile robot datasets over

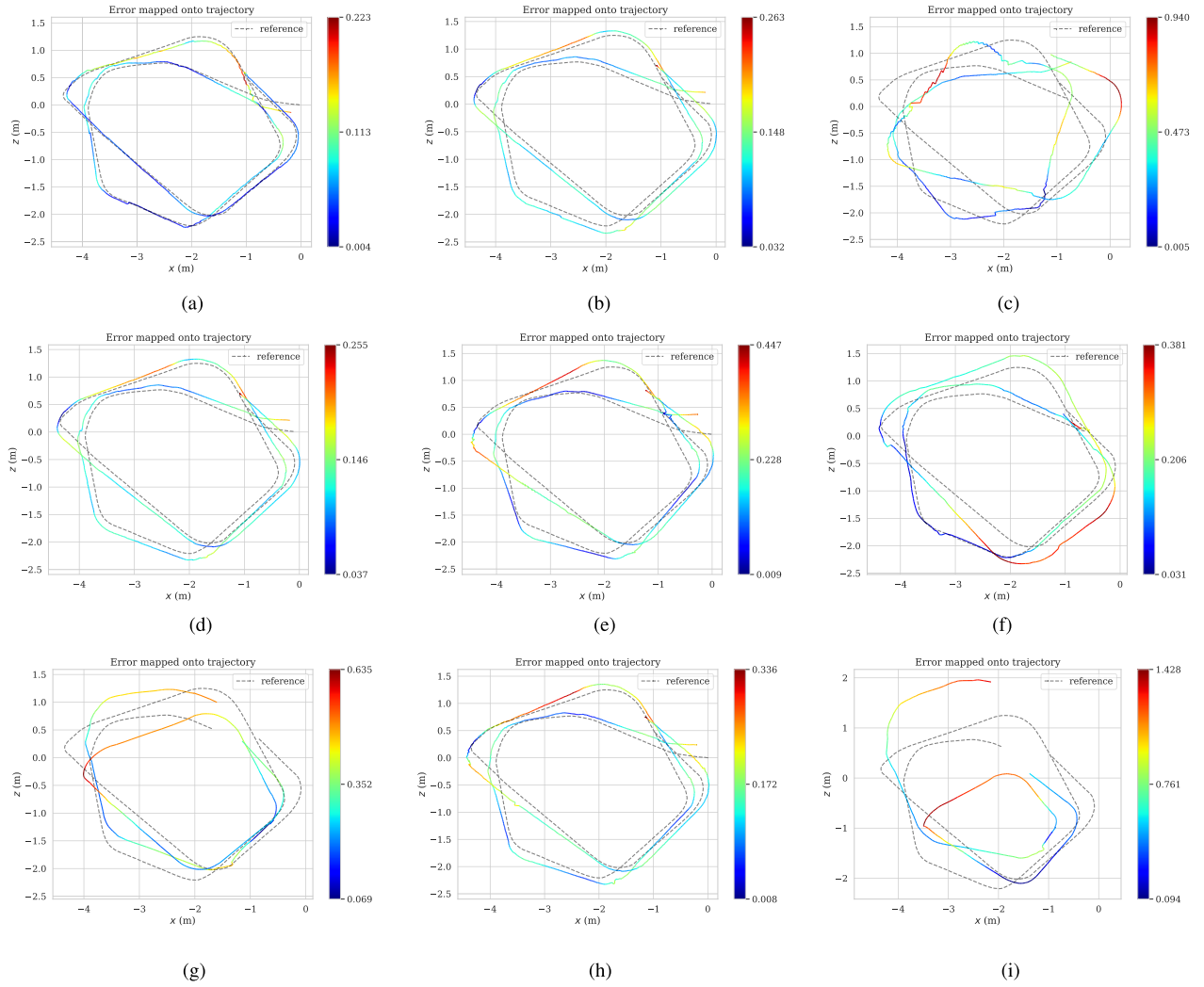


Fig. 7. Color error map trajectories of the proposed and state-of-the-art methods on the *person_walk_1* sequence in the mobile robot dataset over the ground truth. (a) MSSD-SLAM-full. (b) MSSD-SLAM-w/o-PL. (c) MSSD-SLAM-w/o-D. (d) DRG-SLAM. (e) DS-SLAM. (f) Dynamic-VINS. (g) DynaVINS. (h) SG-SLAM. (i) ORB-SLAM3.

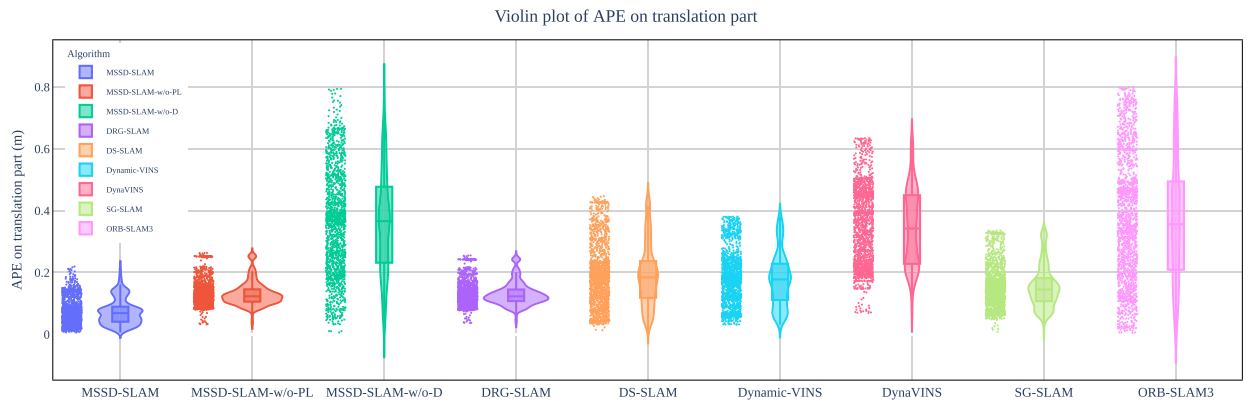


Fig. 8. Violin plots of APE on translation part of sequence *person_walk_1* in the mobile robot dataset.

using the semantic segmentation module alone. In summary, the synergistic operation of multiple modules greatly surpassed the performance of any single module, underscoring the proposed Dynamic Filter's efficacy.

We also analyzed the impact of different thresholds used in our Dynamic Filter on the algorithm's accuracy. The primary thresholds considered are the distance threshold (d_{th}), angle threshold (θ_{th}), and IMU reprojection error threshold (r_{th}).

TABLE VIII
COMPARISON OF THE RMSE OF APE ON TRANSLATION AND RPE ON ROTATION FOR SELF-COLLECTED MOBILE ROBOT SEQUENCES OF ABLATION EXPERIMENTS

Sequences / Dynamic Level	MSSD-SLAM with S		MSSD-SLAM with M		MSSD-SLAM with I		MSSD-SLAM with P		MSSD-SLAM w/o S		MSSD-SLAM w/o M		MSSD-SLAM w/o I		MSSD-SLAM w/o P		MSSD-SLAM full	
	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)	Trans.(m)	Rot.(deg)
<i>person_walk_1</i> / ***	0.3228	1.0343	0.2758	0.8021	0.3848	1.1427	0.3713	1.0327	0.2595	0.7008	0.1631	0.7034	0.1610	0.5995	0.1338	0.5957	0.0833	0.2727
<i>person_walk_2</i> / ***	0.3422	0.7765	0.3540	0.7196	0.4422	0.9515	0.3035	0.9857	0.1825	0.7188	0.2203	0.7733	0.1600	0.5895	0.1524	0.5649	0.1068	0.2563
<i>hold_box_1</i> / **	0.2521	1.0530	0.3088	1.0112	0.3695	0.9729	0.3759	1.0393	0.2688	0.8584	0.1997	0.8327	0.1695	0.7290	0.1672	0.5804	0.0892	0.2924
<i>hold_box_2</i> / ***	0.2755	1.0815	0.2920	0.9818	0.3419	1.0395	0.3817	1.0228	0.2499	0.9112	0.2075	1.0116	0.1673	0.8304	0.1772	0.7135	0.1036	0.4077
<i>hold_box_3</i> / ***	0.2610	0.6768	0.3390	0.4784	0.4442	0.5690	0.3268	0.6762	0.2781	0.4633	0.2157	0.3884	0.1954	0.3906	0.1831	0.3813	0.1117	0.2168
<i>Average</i>	0.2907	0.9245	0.3139	0.7986	0.3965	0.9351	0.3519	0.9513	0.2477	0.7305	0.2013	0.7419	0.1706	0.6278	0.1627	0.5671	0.0989	0.2892

TABLE IX
IMPACT OF IMU REPROJECTION ERROR THRESHOLD (r_{th}) ON RMSE

r_{th}	1	2	3	4
RMSE (m)	0.5183	0.2642	0.3111	1.0433

TABLE X
IMPACT OF DISTANCE (d_{th}) AND ANGLE (θ_{th}) THRESHOLDS ON RMSE

$d_{th} \backslash \theta_{th}$	1	2	3	4
1	0.2894	0.3337	2.6201	1.4985
2	0.5671	0.1188	0.1552	0.9596
3	0.1358	0.0981	0.1311	1.3029
4	1.1406	0.9790	2.0005	2.2411

a) *IMU reprojection error threshold (r_{th}):* We evaluated the IMU reprojection error threshold by varying r_{th} from 1 to 4. For values greater than 4, the algorithm failed to maintain accurate localization. The results, summarized in Table IX, indicate that the optimal r_{th} is 2, yielding the lowest RMSE of 0.2642 m. Therefore, we fixed r_{th} at 2 for subsequent tests.

b) *Distance threshold (d_{th}) and angle threshold (θ_{th}):* With r_{th} fixed at 2, we varied d_{th} and θ_{th} from 1 to 4. For values exceeding 4, the algorithm either lost track or experienced significant drift. The results, presented in Table X, show that the optimal thresholds are $d_{th} = 3$ and $\theta_{th} = 2$, resulting in the lowest RMSE of 0.0981 m.

By systematically testing these thresholds, we established that the optimal values for our Dynamic Filter are $d_{th} = 3$, $\theta_{th} = 2$, and $r_{th} = 2$. These values minimize localization errors and enhance the robustness of our SLAM system.

C. Algorithm Runtime Analysis

To evaluate the computational efficiency of the algorithm, we averaged the time consumption of each main module of the proposed algorithm on the TUM and our collected datasets. The time consumption was measured using the aforementioned personal computer. Table XI shows the results, with an average time consumption of 41.3 ms for the extraction of features, 46.9 ms for the semantic segmentation module, 13.5 ms for the Dynamic Filter, and an average total tracking time of 70.9 ms per frame with the additional steps on our dataset. The average time consumption for local inertial BA was 85.8 ms.

TABLE XI
AVERAGE TIME COST OF DIFFERENT PARTS

Dataset	Feature Extraction (ms)	Semantic Segment (ms)	Dynamic Filter (ms)	Total Tracking (ms)	Local BA (ms)
TUM	46.9	48.9	12.4	81.2	88.9
Self-collected	41.3	46.9	13.5	70.9	85.8

Our SLAM technique enhanced efficiency by 38% compared with DRG-SLAM, which took 115.5 ms per frame, enabling near real-time operation. However, MSSD-SLAM remains more time-consuming than DS-SLAM, which averages 51.3 ms per frame. The primary computational costs in our algorithm are due to the extraction of line and plane features and the deep-learning-based semantic segmentation module.

Future work will aim to enhance efficiency by adopting more efficient feature extraction algorithms, using lighter segmentation models, or leveraging GPU acceleration for parallel processing. These optimizations are intended to reduce the computational burden and improve the real-time performance of the algorithm.

VII. CONCLUSION

This article presents MSSD-SLAM, a multifeature RGB-D inertial SLAM system designed for structured dynamic indoor environments. The system leverages point, line, and plane features within the environment, employing the Manhattan world assumption to impose structured constraints. We have introduced the Dynamic Filter, which integrates semantic segmentation, structured constraints, IMU observations, and forward/backward projection to effectively handle dynamic environments. This filter computes dynamic probabilities for landmarks and subsequently fuses and updates them using binary Bayesian filter. Landmarks that surpass a predetermined threshold in dynamic probability are disregarded for further processing, while those with lower dynamic probability serve as weights, improving robustness during subsequent optimization. The algorithm's performance is assessed on both the TUM dataset and real-world scenarios. The results underscore that our algorithm achieves superior localization accuracy, outperforming other state-of-the-art dynamic visual SLAM methods in managing both recognized and unrecognized dynamic objects. Moreover, our algorithm exhibits notable robustness, particularly in highly dynamic and textureless scenes.

In the future, we aim to boost the efficiency of the algorithm and broaden its capabilities to include dynamic object tracking and prediction. Furthermore, our plan includes developing object-level semantic mapping and enhancing scene comprehension and presentation.

REFERENCES

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [2] R. Mur-Artal and J. D. Tardos, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.
- [3] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
- [4] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [5] G. Zhang, J. H. Lee, J. Lim, and I. H. Suh, "Building a 3-D line-based map using stereo SLAM," *IEEE Trans. Robot.*, vol. 31, no. 6, pp. 1364–1377, Dec. 2015.
- [6] Y. He, J. Zhao, Y. Guo, W. He, and K. Yuan, "PL-VIO: Tightly-coupled monocular visual-inertial odometry using point and line features," *Sensors*, vol. 18, no. 4, p. 1159, Apr. 2018.
- [7] R. Gomez-Ojeda, F.-A. Moreno, D. Zuniga-Noel, D. Scaramuzza, and J. Gonzalez-Jimenez, "PL-SLAM: A stereo SLAM system through the combination of points and line segments," *IEEE Trans. Robot.*, vol. 35, no. 3, pp. 734–746, Jun. 2019.
- [8] D. Zou, Y. Wu, L. Pei, H. Ling, and W. Yu, "StructVIO: Visual-inertial odometry with structural regularity of man-made environments," *IEEE Trans. Robot.*, vol. 35, no. 4, pp. 999–1013, Aug. 2019.
- [9] X. Zhang, W. Wang, X. Qi, Z. Liao, and R. Wei, "Point-plane SLAM using supposed planes for indoor environments," *Sensors*, vol. 19, no. 17, p. 3795, Sep. 2019.
- [10] Y. Li, N. Brasch, Y. Wang, N. Navab, and F. Tombari, "Structure-SLAM: Low-drift monocular SLAM in indoor environments," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6583–6590, Aug. 2020.
- [11] Y. Li, R. Yunus, N. Brasch, N. Navab, and F. Tombari, "RGB-D SLAM with structural regularities," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 11581–11587.
- [12] F. Shu, J. Wang, A. Paganì, and D. Stricker, "Structure PLP-SLAM: Efficient sparse mapping and localization using point, line and plane for monocular, RGB-D and stereo cameras," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 2105–2112.
- [13] D.-H. Kim and J.-H. Kim, "Effective background model-based RGB-D dense visual odometry in a dynamic environment," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1565–1573, Dec. 2016.
- [14] S. Li and D. Lee, "RGB-D SLAM in dynamic environments using static point weighting," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2263–2270, Oct. 2017.
- [15] Y. Sun, M. Liu, and M. Q.-H. Meng, "Motion removal for reliable RGB-D SLAM in dynamic environments," *Robot. Auto. Syst.*, vol. 108, pp. 115–128, Oct. 2018.
- [16] Z.-J. Du, S.-S. Huang, T.-J. Mu, Q. Zhao, R. R. Martin, and K. Xu, "Accurate dynamic SLAM using CRF-based long-term consistency," *IEEE Trans. Vis. Comput. Graph.*, vol. 28, no. 4, pp. 1745–1757, Apr. 2022.
- [17] S. Song, H. Lim, A. J. Lee, and H. Myung, "DynaVINS: A visual-inertial SLAM for dynamic environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11523–11530, Oct. 2022.
- [18] C. Yu et al., "DS-SLAM: A semantic visual SLAM towards dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 1168–1174.
- [19] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, "DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes," *IEEE Robot. Autom. Lett.*, vol. 3, no. 4, pp. 4076–4083, Oct. 2018.
- [20] B. Bescos, C. Campos, J. D. Tardos, and J. Neira, "DynaSLAM II: Tightly-coupled multi-object tracking and SLAM," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5191–5198, Jul. 2021.
- [21] D. Yang et al., "DRE-SLAM: Dynamic RGB-D encoder SLAM for a differential-drive robot," *Remote Sens.*, vol. 11, no. 4, p. 380, Feb. 2019.
- [22] X. Yuan and S. Chen, "SaD-SLAM: A visual SLAM based on semantic and depth information," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Las Vegas, NV, USA, Oct. 2020, pp. 4930–4935.
- [23] Y. Liu and J. Miura, "RDS-SLAM: Real-time dynamic SLAM using semantic segmentation methods," *IEEE Access*, vol. 9, pp. 23772–23785, 2021.
- [24] J. Liu, X. Li, Y. Liu, and H. Chen, "RGB-D inertial odometry for a resource-restricted robot in dynamic environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 9573–9580, Oct. 2022.
- [25] H. Yin, S. Li, Y. Tao, J. Guo, and B. Huang, "Dynam-SLAM: An accurate, robust stereo visual-inertial SLAM method in dynamic environments," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 289–308, Feb. 2023.
- [26] Y. Wang, K. Xu, Y. Tian, and X. Ding, "DRG-SLAM: A semantic RGB-D SLAM using geometric features for indoor dynamic scene," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 1352–1359.
- [27] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.
- [28] X. Zuo, X. Xie, Y. Liu, and G. Huang, "Robust visual SLAM with point and line features," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 1775–1782.
- [29] C. Sun, X. Wu, J. Sun, C. Sun, and L. Dong, "Robust pose estimation via hybrid point and twin line reprojection for RGB-D vision navigation," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–19, 2022.
- [30] H. Wei, F. Tang, Z. Xu, and Y. Wu, "Structural regularity aided visual-inertial odometry with novel coordinate alignment and line triangulation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 10613–10620, Oct. 2022.
- [31] B. Xu, P. Wang, Y. He, Y. Chen, Y. Chen, and M. Zhou, "Leveraging structural information to improve point line visual-inertial odometry," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 3483–3490, Apr. 2022.
- [32] J. Huang, S. Yang, Z. Zhao, Y.-K. Lai, and S. Hu, "ClusterSLAM: A SLAM backend for simultaneous rigid body clustering and motion estimation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 5874–5883.
- [33] J. Huang, S. Yang, T. Mu, and S. Hu, "ClusterVO: Clustering moving instances and estimating visual odometry for self and surroundings," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 2165–2174.
- [34] Z. Pan, J. Hou, and L. Yu, "Optimization RGB-D 3-D reconstruction algorithm based on dynamic SLAM," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.
- [35] S. Cheng, C. Sun, S. Zhang, and D. Zhang, "SG-SLAM: A real-time RGB-D visual SLAM toward dynamic scenes with semantic and geometric information," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023.
- [36] Z. Xing, X. Zhu, and D. Dong, "DE-SLAM: SLAM for highly dynamic environment," *J. Field Robot.*, vol. 39, no. 5, pp. 528–542, Aug. 2022.
- [37] Z. Chang, H. Wu, and C. Li, "YOLOv4-tiny-based robust RGB-D SLAM approach with point and surface feature fusion in complex indoor environments," *J. Field Robot.*, vol. 40, no. 3, pp. 521–534, May 2023.
- [38] Q. U. Islam, H. Ibrahim, P. K. Chin, K. Lim, and M. Z. Abdullah, "MVS-SLAM: Enhanced multiview geometry for improved semantic RGBD SLAM in dynamic environment," *J. Field Robot.*, vol. 41, no. 1, pp. 109–130, Jan. 2024.
- [39] B. Song, X. Yuan, Z. Ying, B. Yang, Y. Song, and F. Zhou, "DGM-VINS: Visual-inertial SLAM for complex dynamic environments with joint geometry feature extraction and multiple object tracking," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.
- [40] S. Wen et al., "Dynamic SLAM: A visual SLAM in outdoor dynamic scenes," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2023.
- [41] S. Wen, S. Tao, X. Liu, A. Babiarz, and F. Richard Yu, "CD-SLAM: A real-time stereo visual-inertial SLAM for complex dynamic environments with semantic and geometric information," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–8, 2024.
- [42] C. Feng, Y. Taguchi, and V. R. Kamat, "Fast plane extraction in organized point clouds using agglomerative hierarchical clustering," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 6218–6225.
- [43] Y. Zhou, L. Kneip, C. Rodriguez, and H. Li, "Divide and conquer: Efficient density-based tracking of 3D sensors in Manhattan worlds," in *Proc. Asian Conf. Comput. Vis.*, vol. 10115. Cham, Switzerland: Springer, 2017, pp. 3–19.

- [44] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, Atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.
- [45] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal visual object classes (VOC) challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
- [46] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Trans. Robot.*, vol. 33, no. 1, pp. 1–21, Feb. 2017.
- [47] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 573–580.



Cheng Chen received the B.E. and M.S. degrees in mechanical engineering and automation from Beihang University, Beijing, China, in 2019 and 2022, respectively, where he is currently pursuing the Ph.D. degree in robotics.

His research interests include motion planning for wheel-legged robot and exploration strategy for multirobot systems.



Yanan Wang was born in Luoyang, China. He received the B.E. degree in mechatronics engineering from Shenyang Aerospace University, Shenyang, China, in 2016, and the M.S. degree in mechanical engineering from Beihang University, Beijing, China, in 2020, where he is currently pursuing the Ph.D. degree in mechatronics engineering.

His research interests mainly include visual SLAM and robot perception.



Yaobin Tian received the master's and Ph.D. degrees from the School of Mechanical, Electronic and Control Engineering from Beijing Jiao Tong University, Beijing, China, in 2009 and 2015, respectively.

He was a Post-Doctoral Fellow with the Department of Industrial Engineering and Logistics Management, The Hong Kong University of Science and Technology, Hong Kong, from 2017 to 2018. He is currently an Associate Professor with the Institute of Robotics, School of Mechanical Engineering

and Automation, Beihang University, Beijing. His current research interests include robotics, parallel robot, mechanism design, multimode mobile robot, and additive manufacturing.



Jiawei Chen (Member, IEEE) was born in Bengbu, China. He received the B.E. degree in mechanical engineering and automation from the University of Science and Technology Beijing, Beijing, China, in 2014, and the Ph.D. degree in mechanical design and theory from Beihang University, Beijing, in 2022.

He is currently a Post-Doctoral Research Fellow with the School of Mechanical Engineering and Automation, Beihang University. His research interests include the design, plan, and control of the wheel-legged robot.



Kun Xu (Member, IEEE) received the B.Eng. degree in mechanical design and manufacture, the M.Eng. degree in mechanical engineering, and the Ph.D. degree in mechanical design and theory from Beihang University, Beijing, China, in 2005, 2008, and 2012, respectively.

From August 2012 to December 2012, he was a Visiting Researcher with the Department of Mechanical and Manufacturing Engineering, Aalborg University, Aalborg, Denmark. From March 2016 to March 2017, he was a Visiting Scholar with the

Department of Engineering, Aarhus University, Aarhus, Denmark. His research interests include robotics, mechanisms, biomimetic design, and manufacturing.



Xilun Ding received the B.Eng. degree in mechanical engineering from Zhengzhou Institute of Technology, Zhengzhou, China, in 1991, and the M.Eng. degree in mechanical design and the Ph.D. degree in mechatronics and automation from Harbin Institute of Technology, Harbin, China, in 1994 and 1997, respectively.

From September 2000 to September 2001, he was a Post-Doctoral Research Fellow with the School of Computing, Information Systems and Mathematics, South Bank University, London, U.K. His research

interests include the dynamics of compliant mechanical systems and robots, nonholonomic control of pace robots, dynamics and control of aerial robots, and biomimetic robots.